Republic of the Philippines
PAMANTASAN NG LUNGSOD NG MAYNILA
(University of the City of Manila)
General Luna Street corner Muralla Street
Intramuros, Manila

Pothos Compiler

BSCS 3-2 (Group 3)

A.Y 2026-2027

A Compiler presented to the

Computer Science Department

College of Information Systems and Technology Management

In Partial Fulfillment of the Requirements for Degree

Bachelor of Science in Computer Science (BSCS)

Under the Supervision of

Prof. Leisyl C. Mahusay

Submitted by:

Avila, Julien Jamile P.

Bisnan, Leinard Allen B.

Blaza, Charles Aldrin M.

Fernandez, Patrick U.

Francisco, Lj S.



TABLE OF CONTENTS

I. OVERVIEW OF THE LANGUAGE....4
II. STRUCTURE OF THE LANGUAGE....5
III. GENERAL RULES....6
IV. LIST OF RESERVED WORDS....11
V. LIST OF RESERVED SYMBOLS....14
VI. SPECIFIC RULES....16
A. Data Types and Literals....16
A.2 Wharfa....18
A.3 Flo....19
A.4 Trua....21
A.5 Lis....22
A.5.1 Declaration....22
A.5.2 Initialization....23
A.5.3 Indexing....25
A.5.4 Nested lis....25
A.6 Glossare....27
B. Operators....29
C. Identifiers....34
D. Expressions....35
E. Statements and Code Blocks....36
F. Comments....38
G. Variables and Constants....39
H. Assignment and Augmented Assignment....40
I. Built-In Functions....41
I.1 prin()....41
I.2 inpub()....41
I.5 formo()....43
I.6 Display Forms....44
J. Conditional Statements....45
K. Looping Statements....47
K.1 Cycla Loop....47
K.2 Lood Loop....48
L. Loop Control Statements....50
M. Functions....51
M.1 Function Declaration....51
M.2 Parameters....53



M.3 Return Values..... 54
M.4 Function Calls..... 55
M.5 Recursion.....56
M.6 Scope and Visibility.....56
N. Object Removal.....58
VII. REGULAR DEFINITIONS..... 58
VIII. REGULAR EXPRESSION.....62
A. Reserved Words..... 62
B. Reserved symbols..... 63
C. Reserved Literals..... 65
IX. DELIMITERS.....66
X. TRANSITION DIAGRAM..... 70
A. Reserved Words..... 70
B. Reserved Symbols.....72
C. Identifiers..... 74
D. Integer and Float Literals.....75
E. String Literals and Comments.....76

I. OVERVIEW OF THE LANGUAGE



Pothos is a compact, instructional programming language developed to introduce learners to the essential mechanics of compiler construction. Its name draws from the ancient Greek word for longing or yearning, a fitting root for a language whose central goal is to cultivate a deeper understanding of how programming languages work beneath the surface, rather than simply how to use them. Pothos is built on the syntactic structure of C/C++ and Python, using semicolon-terminated statements, curly-brace-delimited blocks, and explicit function return types. For readability, the reserved-word table also maps Pothos keywords to familiar Python counterparts. Unlike traditional languages that require a dedicated main function, Pothos utilizes a script-like execution model where the source file itself serves as the entry point, executing top-level statements sequentially. This hybrid identity defines Pothos: syntax simple enough to read at a glance, yet explicit enough that every rule of the grammar is visible and intentional.

What further sets Pothos apart is its own distinct character naming. Pothos uses thematic, Greek-inspired reserved words. Some keywords are invented or stylized to support the language's identity while still teaching foundational programming concepts. Keywords like wharfa for integers, strina for strings, belial for conditionals, and reor for returns give the language a unique linguistic personality. This design choice gives Pothos a creative identity while still teaching foundational concepts. Furthermore, Pothos embraces automatic type inference for variables and lists, allowing learners to focus on logic rather than verbose type declarations, while still supporting explicit return types for functions. Unique syntactic elements, such as single-line (--) and multi-line (/-- ... --/) comments, alongside built-in I/O functions like prin() and inpub(), expose distinct stages of how a compiler reads, checks, and executes code.



II. STRUCTURE OF THE LANGUAGE

Main file (main.pths) | compile using pothos.py main.pths, run using main.exe

-- optional comments <function_def>/<statement> <function_def>/<statement>/ λ -- optional comments

msg = "Hello, World!";
prin(msg);
 	
wharfa add(wharfa a, wharfa b){
  reor a + b;
}
 	
prin(add(3, 4));

In the Pothos language, every program follows a structured, script-like format to maintain consistency and readability. Unlike traditional languages that require a dedicated main function, the source file itself (.pths) serves as the entry point, executing top-level statements sequentially from top to bottom. Single-line comments are indicated by --, and multi-line comments are enclosed between /-- and --/. Comments describe the code but are ignored during execution. Because Pothos utilizes an automatic type of inference, global and local variables are simply declared by pairing an identifier with a value using an assignment operator (e.g., msg = "Hello";), while lists are initialized using square brackets. Functions are defined by stating an explicit return type (such as wharfa or strina), followed by a unique name and optional parameters enclosed in parentheses. The function body is enclosed in curly brackets {}, where local declarations and executable statements are written, each terminated by a semicolon, and the keyword reor returns a value and exits the function. Because blocks are delimited by curly brackets instead of indentation, and every statement is explicitly terminated, this structure keeps .pths programs organized, readable, and easy to maintain.



III. GENERAL RULES

Pothos is a compiled general-purpose programming language that supports datatypes, console input/output (IO), arithmetic operations, etc. Rules are listed below in topical arrangement.

1. The source code is a file written in plain ASCII text, the file uses the **.pths** extension.
    a. The entry point of a program is the file itself; statements are executed in order from top to bottom.
    b. The Pothos character set is ASCII. Non-ASCII characters (e.g., Ω, ω, emoji, accented letters) are lexical errors in identifiers, reserved words, symbols, and numeric literals; they are permitted only as data inside a strina literal or a comment.
    c. To run the program, you must compile it first using the pothos compiler. Then run the **exe** program generated.
        i. **pothos.py main.pths**
        ii. **helloworld.exe**
    d. Definitions register in the symbol table in top-to-bottom execution order.
        i. Usable variables and functions must be declared first to be used (before the call executes), and a top-level call requires an earlier definition in the source file.
        ii. For names read inside a function body, registration is judged when the call executes.
    e. Functions can access global variables defined anywhere, provided the variable has been declared by the time the function is called.
        i. This is checked when the call executes: the name must already be in the global symbol table at that moment, in addition to their own parameters and local variables.
    f. A program must contain at least one function definition or statement.
        i. An empty file, or a file whose only contents are comments and whitespace, provides no definition or statement and is a syntax error.
2. Identifiers are case-sensitive names of variables, constants, functions, and parameters.



a. Global variable names, constant names, and function names are unique program-wide among active identifiers.
    i. A name freed by deleo may be reused for a new non-constant variable.
b. Function parameter names are local to the function in which they are declared
c. Identifier names cannot be named after existing reserved keywords.
d. Identifier names have a maximum length of 20 characters, with a minimum of one character for variables and three characters for functions.
e. Identifier names must start with a letter and contain only alphanumeric characters and underscores.

3. A code block is a logical grouping of executable statements delimited by curly braces { }.
    a. Code blocks will execute exclusively under three conditions:
        i. when activated by a conditional statement,
        ii. when activated by a looping statement, or
        iii. when invoked through a function call.
    b. Curly braces at statement level may appear only as the body of a conditional statement, a looping statement, or a function declaration.
        i. A bare block (curly braces with no belial, belsa, elsa, cycla, lood, or function header before them) is a syntax error.
        ii. Curly braces appear in three non-interchangeable contexts:
            1. At statement level they delimit code blocks and may follow only a belial, belsa, elsa, cycla, lood, or function header
            2. Inside an expression they open or close a glossare literal
            3. And inside a formo() format string they mark placeholders as interpolation syntax, neither glossare literals nor code blocks.
4. Any identifier that links to a value is called a variable.
    a. Variable types are automatically inferred from their assigned values at declaration and are immutable; they cannot be recast.
        i. i. Value errors are detected in two stages(1–2). Syntax errors form a third, earlier class detected at compile time before any semantic check(3).
            1. Compile time: the compiler rejects the program before it runs, on textual checks over the top level and over each function's own



parameters and locals: type change on reassignment, use of an identifier never declared at top level or never declared within its own function body, argument/parameter count or type mismatch, wrong return type, element-type mismatch on index/entry assignment, any write rooted at a constant identifier, and any operator operand-type mismatch whose operands are statically typed (literals, declared identifiers, parameters, and calls whose return type is declared).

   a. Names read inside a function body that belong to the outside world (globals, other functions) are not checked here.

2. Run time: the program starts and crashes at the moment the bad value is produced or the missing registration is touched, such as use, at the moment the call executes, of a global variable or function whose declaration has not yet been registered, division or modulo by zero, out-of-range lis index on read or write, missing glossare key on read or write, type-violating entry augmented assignment, an operator operand-type mismatch whose operand type is fixed only at run time (a lis element of a run-time-typed lis, or a glossare entry value), invalid input to innum()/inflo(), recursion deeper than 999.
3. Syntax errors (compile time, before either semantic stage): the lexer and the syntax analyzer reject the program before any semantic check or execution: invalid tokens, escape sequences, or casings; unbalanced parentheses, brackets, or braces; a missing semicolon; a bare block; empty lis or glossare literals; a trailing comma before a closing delimiter; and every form the specific rules assign to the lexer or the parser.

ii. Type keywords (wharfa, flo, strina, trua, lis, glossare) appear only in function headers, parameter lists, and in the blank boolean declaration. A type keyword in any other declaration is a syntax error.



   b. A variable is declared by pairing an identifier with a value using an assignment operator, except that a boolean may be declared with **trua** <identifier>;, in which case it defaults to **Inaccu**.
      i. Anything that can return a value can be placed instead of an actual value as long as:
         1. That anything guarantees to return a value.
         2. The value returned matches the variable type.

5. Any identifier that links to a reusable block of code is called a function.
   a. Function definitions are initiated with a return data type, which is immediately followed by the unique name and a comma-separated list of parameters enclosed in parentheses, while the executable block itself must be delimited by curly braces { }.
   b. To invoke a specific function, its unique name must be stated along with any required arguments separated by commas and contained within parentheses ().
   c. Function parameters are not required but must be separated by commas (**,**) if there exist multiple.
   d. To exit the function, it must return a value using the keyword '**reor**'.
      i. A bare reor; with no value is a syntax error.
   e. The built-in functions are always available and their names are reserved: **prin()**, **inpub**(), **formo**(), **innum**(), **inflo**().
6. Reserved words are written only in lowercase letters, except the truth literals **Trua** and **Inaccu**, which are capitalized and case-sensitive.
7. Comments are ignored by the compiler.
   a. A single-line comment starts at -- and ends at a newline character or, on the last line of the file, at the end of the file.
   b. A multi-line comment is indicated by /-- at the start and ends with --/.
8. A statement is a group of reserved words, literals, values, operators and functions working together to perform one action.
   a. Simple statements end with a semicolon(;)
   b. Control-flow statements (belial, belsa, elsa, cycla, lood) end with their closing brace without a semicolon;



c. Function declarations end with }.

d. A bare expression that is not a function call is not a statement and is a syntax error, even when a semicolon follows it.

   i. A function call is a statement. Written alone it is a simple statement and ends with a semicolon.

e. The semicolon marks the end of a simple statement:

   i. A simple statement may span several lines, and several simple statements may share one line.

9. Supported data types are **strina**, **wharfa**, **flo**, **trua**, **lis**, and **glossare**.
   a. A wharfa literal is an unsigned sequence of digits capped at 8 significant digits.
      i. The largest wharfa value is 99999999
      ii. A negative integer is formed by applying the unary minus operator to a **wharfa** literal. The lexer does not include - inside **wharfa_lit**.
      iii. Unnecessary leading zeros are discarded and do not count toward the cap.
         1. 0 → 0
         2. 0100 → 100
         3. 00 → 0
         4. 000000001 → 1
   b. A **flo** literal is an unsigned decimal number with the integer part capped at 8 significant digits and the fractional part capped at 4 digits.
      i. A negative **flo** is formed by applying the unary minus operator to a **flo** literal. The lexer does not include - inside **flo_lit**.
      ii. Unnecessary leading zeros in the integer part are discarded when the value is formed. Trailing zeros in the fractional part never change the value. Only prin() and formo() trim trailing zeros at display time.
         1. 0.0000 → value 0.0000, printed as 0.0
         2. 0100.50 → value 100.5000, printed as 100.5
   c. Strings encompass any ASCII character, though embedding the enclosing delimiter and the escape characters requires an escape sequence such as: " → \", \ → \\
      i. Strings also have special escape sequences denoting newline \n, null character \0, \" double quote, \' single quote, \\ backslash, \t horizontal tab.
   d. Booleans are variables that can only have two states.
      i. Declaring it blank will automatically set it to **Inaccu** (false) by default.
10. An array or its equivalent, **lis**, is declared by assigning a list of elements enclosed in square brackets([]) to a variable name. Its element type is inferred from its contents.



   a. Each lis value's element count is immutable.
      i. Element values are editable in place by index assignment.
      ii. Rebinding a variable to a new lis of different length replaces the value.
   b. A lis can have at most 3 dimensions.
      i. A 4th level or higher is a semantic error.
   c. Array elements are accessed with square brackets using a zero-based index.
11. The assignment operator is =. The comparison operators are ==, !=, <, <=, >, >=.
12. Arithmetic operators are symbols that perform changes within variables and values. The arithmetic operators included are **+, -, *, /,** and %. The symbols represent the operation accordingly.
   a. wharfa / wharfa division gives a whole number output.
      i. If either side is a flo, the result is a flo rounded to 4 decimals.
   b. The % operator only works on wharfa.
13. A conditional statement is written as '**belial**' (condition) { }, with optional additional '**belsa**' (condition) { } branches and an optional final '**elsa**' { } branch.
14. Logical operators evaluate relations between boolean values using **an** (and), **othan** (or), and **neve** (not).
   a. an and othan take two trua operands while neve takes one.
15. A '**cycla**' (condition) { } loop repeats while the condition is '**Trua**'.
   a. A sequence-form lood(<name> insid<expression yielding lis or strina>){} loop iterates over each element. A direct lis or strina literal is a permitted iterable.
   b. A counted **lood**(<assignment>; <condition>; <assignment>){} loop runs the init once, repeats while the condition is **Trua**, and runs the step after each iteration.
   c. '**paus**'; exits the nearest loop.
   d. '**persis**'; skips to the next iteration.
16. The standalone statement '**pas**'; does nothing.
17. The keyword '**insid**' has two roles: as an operator, it tests whether a value is contained in a **lis** or **strina**.
   a. In a sequence-form **lood** header, '**insid**' introduces the **iterable** (**lis** or **strina**) whose elements the loop visits; the counted form does not use '**insid**'.



IV. LIST OF RESERVED WORDS

COUNTERPART (C / C++/ Python)	FUNCTIONS/DESCRIPTION	List of Reserved Words (Proposed)	REGULAR EXPRESSION	EXAMPLES
1. **Built-in Data Types**				
str	Represents text strings.	strina	(s)(t)(r)(i)(n)(a)	msg = "Hello"; (implicit declaration) strina printx(){ reor "Hi"; } (return type)
int	Represents integer numbers.	wharfa	(w)(h)(a)(r)(f)(a)	count = 10; (implicit declaration) wharfa numx(){ reor 0; } (return type)
float	Represents numbers with decimals.	flo	(f)(l)(o)	price = 19.99; (implicit declaration) flo flox(){ reor 0.0; } (return type)
bool	Represents boolean truth values.	trua	(t)(r)(u)(a)	trua active;
lis	Represents ordered sequences.	lis	(l)(i)(s)	nums = [1, 2, 3]; lis items(){ reor [1]; } (return type)
dict	Represents key-value mappings.	glossare	(g)(l)(o)(s)(s)(a)(r)(e)	data = {"a": 1}; glossare book(){ reor {"a":1}; } (return type)



2. Input / Output Functions				
print()	Outputs text to the console.	prin()	(p)(r)(i)(n)	prin("Hello");
input()	Reads input from the user.	inpub()	(i)(n)(p)(u)(b)	txt = inpub();
format()	Formats string values.	formo()	(f)(o)(r)(m)(o)	s = formo("Sum: {x}");
int(input())	Reads one line of input as a wharfa.	innum()	(i)(n)(n)(u)(m)	n = innum();
float(input())	Reads one line of input as a flo.	inflo()	(i)(n)(f)(l)(o)	f = inflo();
3. Value & Truth Literals				
False	Boolean false value.	Inaccu	(I)(n)(a)(c)(c)(u)	flag = Inaccu;
True	Boolean true value.	Trua	(T)(r)(u)(a)	flag = Trua;
4. Logical Operators				
and	Logical conjunction.	an	(a)(n)	a an b
or	Logical disjunction.	othan	(o)(t)(h)(a)(n)	a othan b
not	Logical negation.	neve	(n)(e)(v)(e)	neve active
5. Structural Control Flow				
if	Conditional execution starts.	belial	(b)(e)(l)(i)(a)(l)	belial (x > 0) {}
elif	Conditional 'else if' branch.	belsa	(b)(e)(l)(s)(a)	belsa (x == 0) {}
else	Conditional fallback branch.	elsa	(e)(l)(s)(a)	elsa {}
for	Iterates over a sequence.	lood	(l)(o)(o)(d)	lood (x insid nums) {} (iterator type)



				lood (x = 0; x < 10; x += 1) {} (counted form)
while	Loops until condition false.	cycla	(c)(y)(c)(l)(a)	cycla (x > 0) {}
break	Exits a loop.	paus	(p)(a)(u)(s)	paus;
continue	Skips to next iteration.	persis	(p)(e)(r)(s)(i)(s)	persis;
pass	No-op statement	pas	(p)(a)(s)	pas;
6. **Structural Definitions**				
return	Returns value from function.	reor	(r)(e)(o)(r)	reor val;
7. **Variable Operations**				
del	Removes objects/references .	deleo	(d)(e)(l)(e)(o)	deleo temp;
const	Declares a constant	const	(c)(o)(n)(s)(t)	const MAX = 100;
8. **Object & Collection Membership**				
in	Checks sequence membership.	insid	(i)(n)(s)(i)(d)	x insid nums



V. LIST OF RESERVED SYMBOLS

List of Reserved Symbols (Existing)	Functions/Descriptions	List of Symbols Words (Proposed)	Examples
Arithmetic Operators			
+	Addition or string concatenation	+	x = 5 + 3; or s = "A" + "B";
-	Subtraction; unary minus handled by parser	-	x = 5 - 3; or y = -5;
*	Multiplication	*	area = w * h;
/	Division	/	p = 10 / 2; (integer result) q = 10.0/2.0 (float result)
%	Modulo	%	r = 10 % 3;
Relational Operators			
<	Less-than comparison	<	belial(x < 10){ }
>	Greater-than comparison	>	belial(y > 5){ }
<=	Less-than-or-equal comparison	<=	belial(z <= 100){ }
>=	Greater-than-or-equal comparison	>=	belial(age >= 18){ }
==	Equality comparison	==	belial(name == "Ana"){}
!=	Inequality comparison	!=	belial(x != 0){}
Assignment and Augmented Operators			
=	Assignment	=	score = 100;
+=	Augmented add-assign	+=	count += 1;
-=	Augmented subtract-assign	-=	lives -= 1;
*=	Augmented	*=	total *= 2;



	multiply-assign		
/=	Augmented divide-assign	/=	avg /= n;
%=	Augmented modulo-assign	%=	rem %= 5;
Grouping Symbols			
(	Open parenthesis -- grouping or function call	(	x = (1 + 2) * 3;
)	Close parenthesis – grouping or function call	)	prin("Hi");
[	Open bracket – list literal or subscript	[	nums = [1, 2];
]	Close bracket – list literal or subscript	]	x = nums[0];
{	Open brace – glossare literal or code block	{	belial(x==1){
}	Close brace – glossare literal or code block	}	data = {"a":1};
Others			
,	Separator for arguments or elements	,	nums = [1, 2, 3];
"	Double quote: opens or closes a strina literal	"	msg = "Hello";
'	Single quote: opens or closes a strina literal	'	msg = 'Hello';
:	Key separator in glossare literals	:	data = {"name": "Ana"};
;	Statement separator	;	x = 5;
--	Comment -- rest of line is ignored	--	x = 5; -- this is a comment
/--	Multi-line comment start	/--	/-- start of comment
--/	Multi-line comment end (a --/ with no opening /-- is an error)	--/	end of comment --/

VI. SPECIFIC RULES

A. Data Types and Literals

Pothos has six built-in data types: **strina** (text), **wharfa** (integer), **flo** (decimal), **trua** (boolean), **lis** (list), and **glossare** (dictionary). Literals are fixed values written in the source code.

A.1 Strina

strina is the text type of Pothos, used for sequences of ASCII characters:

1. A **strina** literal is enclosed in a matching pair of double quotes (") or single quotes (').
    a. Mismatched opening and closing delimiters are an error.
    b. Any unclosed text sequence is a lexical error; the tokenizer absorbs the full line following the opening delimiter as an unclosed **strina** token.
2. A **strina** literal may also be empty: "" or ''.
3. A strina literal has no maximum length.
4. Strings (strina) may contain any ASCII character, as well as non-ASCII characters. However, the enclosing quotation mark and the backslash (\) must be escaped when included as literal characters.
    a. The enclosing quotation mark must be escaped using \' for single-quoted strings or \" for double-quoted strings. A literal backslash must be escaped using \\. The following escape sequences are supported:
        i. \n denotes newline
        ii. \\ denotes backslash
        iii. \' denotes single quote
        iv. \" denotes double quote
        v. \t denotes horizontal tab
        vi. \0 denotes null character
    b. Enclosing string literals in single quotes permits unescaped double quotes internally, and vice versa.
    c. Non-ASCII characters are permitted only inside strina literals and comments; outside them they trigger lexical errors.
    d. \' and \" are valid escapes in either quoting style; a raw opposite quote and its escaped form denote the identical character
    e. Escape sequences are resolved at lexical time; the value remembers neither its delimiter style nor any backslash.



f. At assignment, comparison, prin, formo, and collection display, quote characters inside a value are plain data. Quotes act as delimiters only inside the source literal.

5. The enclosing quotation mark must be escaped to appear inside the string literal (\" or \').
    a. An unescaped enclosing quotation mark terminates the string literal. If the succeeding character is not a valid delimiter, a lexical error occurs.
        i. For example, "Compiler "pothos" comment" produces a lexical error because the string "Compiler " terminates at the second double quotation mark, and the succeeding character p is not a valid delimiter.
6. An unescaped newline cannot appear inside a strina literal. The escape sequence \n must be used to represent a newline character.
    a. A newline encountered before the closing quotation mark causes a lexical error.
        i. For example, a strina literal containing "a" followed by a newline and then "b" produces a lexical error. The lexer stops and reports an "unterminated string" error.
7. Two **strina** values may be joined with the + operator.

Syntax
"<characters>" or '<characters>'

Valid Examples	**Invalid Examples**
title = "Pothos";	bad1 = "Hello; (unterminated string)
double_empty = ""; single_empty = '';	bad2 = 'AB"; (mismatched delimiters)
message = "Line one\nLine two";	bad3 = "Newline here"; (unescaped newline)
quote = "She said,\"Hi.\"";	bad4 = "Hi\x"; (invalid escape sequence)
pothos = "Compiler 'pothos' ";	comment = "Compiler "pothos" comment" ; (missing escape characters)
comment = ' Compiler "pothos" ' ;	comment = 'Compiler 'pothos' comment' ; (missing escape characters)



greet = "hello world";	name= Ω; (non-ASCII outside a string)
greet="hello Ω";	

A.2 Wharfa

wharfa is the integer type of Pothos, written without decimal points or fractions:

1. A **wharfa** literal is an unsigned sequence of digits from 0 to 9.
2. A negative wharfa value is produced by the unary minus (-) operator before the digits; positive literals are written without a plus sign.
    a. A leading plus sign (+) is not permitted as a unary operator anywhere in an expression. Its use results in a syntax error.
        i. For example, z = +25;, x = 3 * +2;, and f(+1) produce syntax errors because unary plus is not supported.
3. Leading zeros are discarded and do not count toward the digit limit (0100 -> 100).
4. The maximum size of a wharfa literal is 8 significant digits, with an allowed value range of -99999999 to 99999999.
    a. A numeric sequence exceeding 8 significant digits results in a syntax error rather than a lexical error.
        i. The tokenizer accepts the first 8 significant digits as one wharfa_lit token and processes the remaining digits as one or more separate wharfa_lit tokens.
        ii. For example, 123456789 is tokenized into two consecutive wharfa_lit tokens: 12345678 and 9.
        iii. The syntax analyzer detects the error when it encounters two consecutive wharfa_lit tokens without a valid operator or delimiter between them.
    b. The tokenizer counts significant digits beginning with the first non-zero digit. Leading zeros do not contribute to the 8-significant-digit limit.
        i. The tokenizer absorbs all leading zeros and up to 8 significant digits into the first wharfa_lit token. Any surplus digits are processed as separate tokens.
        ii. For example, 000123456789 is tokenized into 00012345678 and 9, causing the syntax analyzer to report an error due to consecutive numeric literals.
5. Decimal points, commas, underscores, spaces, and exponent symbols are not allowed inside a wharfa literal.
    a. A decimal point (.) immediately following digits is permitted only when forming a valid flo literal with at least one digit after the decimal point.



   i. For example, 1.5 is recognized as a flo literal, while 5. produces a lexical error because no digit follows the decimal point.
  b. Underscores (_) and exponent letters (e or E) are not permitted within or immediately after numeric literals and result in lexical errors.
   i. For example, 5_000 produces a lexical error because an underscore cannot follow a numeric literal.
   ii. Similarly, 1e5 produces a lexical error because exponent notation is not supported and the letter e cannot immediately follow a numeric literal.
  c. Commas and spaces are recognized as separate tokens or delimiters. Their improper use results in syntax errors.
   i. For example, 1 000 and 1,000 do not represent valid single wharfa literals.
6. A **wharfa** value may only be assigned to a **wharfa** variable or used where an integer is expected.
  a. Using a **wharfa** where another type is expected is an error.
  b. The exception is that a **wharfa** may operate alongside a **flo** under promotion; the result is a **flo** value.

Valid Examples	Invalid Examples
a = 0; b = 100;	x = 100000000; (exceeds 8 significant digits)
c = -99999999;	y = 1,000; (comma not allowed)
d = 0100;	z = +25; (plus sign not allowed)
e = 50;	w = 1.5; (decimal point not allowed)
f= 000000001;	

A.3 Flo

flo is the decimal number type of Pothos, used for numeric values with a fractional part:

1. A **flo** literal is an unsigned sequence of digits, a single decimal point (.), then a second sequence of digits; at least one digit is required on both sides of the point.
  a. A decimal point missing a digit on either side is a lexical error.
2. A negative **flo** value is produced by the unary minus (-) operator before the digits; positive literals are written without a plus sign.
  a. A leading '+' is an error.



3. Leading zeros in the integer part are discarded and do not count toward the digit limit.
    a. Trailing zeros in the fractional part belong to the literal as typed (they count toward the 4-digit cap) and remain in the stored fixed-point value. They are trimmed only in the displayed output of prin() and formo().
4. The integer part of a flo literal is limited to 8 significant digits, while the fractional part is limited to 4 digits. The allowed range is -99999999.9999 to 99999999.9999.
    a. Exceeding the 8-significant-digit limit in the integer part results in a syntax error.
        i. The tokenizer absorbs all leading zeros and up to 8 significant integer digits. Any surplus integer digits are processed as separate tokens.
    b. A flo literal containing more than 4 fractional digits is truncated to 4 decimal places without rounding or generating an error.
        i. For example, 1.23456 is recognized as a single flo literal with the value 1.2345. The fifth and succeeding fractional digits are discarded.
    c. Truncation applies only to flo literals during lexical analysis. Arithmetic results exceeding 4 decimal places follow the round-half-up rule specified for arithmetic operations.
5. Exponent symbols, commas, underscores, and spaces are not allowed inside a **flo** literal.
    a. '_' and exponent letters after digits are lexical errors; a '.' that does not complete a **flo literal** is a lexical error.
    b. Commas and spaces are parser errors.
6. A **flo** value is stored as a fixed-point decimal with exactly 4 fractional digits; arithmetic results are rounded half-up to 4 fractional digits.
    a. Equality (==, !=) and ordering (<, <=, >, >=) compare the stored fixed-point values and produce a **trua** value.
7. When printed by **prin**() or interpolated by **formo**(), a **flo** displays up to 4 fractional digits, trimming trailing zeros but always keeping at least one fractional digit.
8. A **flo** value may only be assigned to a **flo** variable or used where a **flo** is expected.
    a. Using a **flo** where another type is expected is an error.
    b. Mixing **wharfa** and **flo** operands is permitted only inside operator evaluation under the promotion rule.
        i. No implicit conversion exists anywhere else. Assignment, augmented assignment, argument passing, and return values remain exact-type.



Syntax
<digits>.<digits>

Valid Examples	Invalid Examples
a = 0.0; b = 19.99;	x = 5.; (no digit after the point)
b = 0.5;	y = .5; (no digit before the point)
c = -99999999.9999;	z = +2.5; (plus sign not allowed)
e = 7.0 / 2.0; (→ 3.5)	w = 1.23456; (fractional cap exceeded)
price = 19.99; prin(price);	v = 123456789.1; (integer cap exceeded)
t = 1.0 + 1.5;	u = 1.5e2; (exponent symbol not allowed)
result = 1.0 + 1.5;	s = 5.0 % 2.0; (modulo not defined for flo)
quotient = 5.0 / 2.0;	
t = 1 + 1.5;	

A.4 Trua

trua is the boolean type of Pothos, holding exactly one of two truth states:

1. The type keyword is lowercase: **trua**.
2. The truth literals are capitalized and case-sensitive: **Trua** (true) and **Inaccu** (false).
    a. Any other casing is an error.
3. Numeric substitutes such as 1 or 0 are not valid boolean literals.
4. A **trua** variable may be explicitly declared without initialization (**trua** <identifier>;), in which case it defaults to **Inaccu**.
5. Logical expressions and relational expressions produce **trua** values.

Syntax
trua <identifier>;

Valid Examples	Invalid Examples



active = Trua; inactive = Inaccu;	active = true; (invalid literal)
trua flag;	locked = 0; (numeric substitute)
result = 3 > 2;	flag = Trua (missing semicolon)
	trua flag= Trua; (typed declaration)

A.5 Lis

lis is the fixed-length, ordered sequence type of Pothos. A lis must contain at least one element. Each lis value's length is fixed at creation and can never change. What is constant is the length, what is editable is the element.

A.5.1 Declaration

1. A lis variable is declared by assigning a non-empty lis literal or a lis-yielding expression to an identifier; lis has no type-keyword declaration form.
    a. **lis** nums; is an error; only **trua** has a blank declaration.
    b. Using a **lis** identifier before its declaration is an error.
    c. An empty lis literal [] is not permitted and reports a syntax error when encountered.
2. The element type is inferred from the elements at declaration and is part of the variable's immutable type.
    a. For a lis literal, the element type is determined by its first element. All succeeding elements must be compatible with that type, otherwise, an error occurs.
    b. Reassignment must supply a non-empty lis of the same element type or an error occurs.
    c. Fixed length is per value.
        i. The new value's length may differ from the length of the value it replaces.
        ii. Length belongs to the value which fixes only the element type.
    d. Since an empty lis is prohibited, every lis variable has a determinable element type upon initialization.
3. **Lis-yielding expressions** are:
    a. another **lis** variable, or a call to a function whose return type is **lis**.
    b. A lis-yielding expression must produce a lis containing at least one element.



c. An empty lis cannot be used as an initial value, reassignment value, argument, or return value.

4. A function may declare **lis** parameters and a **lis** return type. A parameter needs no literal in the header. It is initialized by the argument at call time.
   a. A lis argument must contain at least one element.
   b. The element type must remain compatible with the parameter's established type requirements.

Syntax
<identifier> = [<value>, ...];
<identifier> = <lis variable | lis-returning call>;

Valid Examples	**Invalid Examples**
nums=[1,2,3];	lis nums; (illegal declaration)
copy=nums;	prin(nums[0]); (undeclared use)
words=labels();	nums=[1,2]; nums=["a","b"]; (type mismatch)
	empty=[]; (empty list)

A.5.2 Initialization

1. A **lis** literal is enclosed in square brackets [], elements separated by commas.
   a. A lis must contain at least one element upon creation. An empty lis literal [] is a syntax error.
   b. A missing closing bracket, missing separator, or trailing comma results in a syntax error.
   c. The lexer recognizes [and] as separate valid tokens.
      i. For example, nums=[]; produces a syntax error because a lis requires at least one element.
2. Each element is an expression, evaluated from left to right at creation; the stored element is the resulting value.
   a. Elements may be literals, arithmetic expressions, variables, or function calls.
3. All elements must yield the same data type; that common type becomes the element type of the lis.
   a. Mixed element types are an error.
   b. Any of the six types may serve as the element type, including lis (nested lis) and glossare.



i. Homogeneity for glossare elements is checked at the type level only. No key-set comparison is performed at creation.

4. Nested lis is Pothos's multidimensional form: every element must be of type lis, and all inner lis values must share one element type.
    a. Every inner lis must contain at least one element. Empty inner lis literals are not permitted.
    b. For example, [[1,2],[3,4]] is valid because both inner lis values contain elements of the same type.
    c. For example, [[1],[]] is a syntax error because the second inner lis is empty.
    d. Nested lis values may have different lengths, provided their element types are compatible.
5. Length equals the element count at creation and is fixed for that value.
    a. No statement or operator adds, removes, shifts, or resizes elements.
    b. Element values are editable in place by index assignment.
    c. Reassignment links the variable to a whole new lis value and does not resize the old one.
    d. Every lis must have a length of at least one.
    e. Reassignment must preserve the established element type but may change the length of the newly assigned lis.
6. Strina elements must be quoted strina values, using either quotation delimiter.
7. Assignment links names to values without copying:
    a. after copy=nums;, both names link to one lis value, and an index assignment at any depth through either is visible through both.
    b. Whole-value reassignment (copy=[9,9];) rebinds only copy. The same holds for glossare and for lis/glossare arguments.

Syntax
<identifier> = [<expr>, <expr>, ... <expr>];

Valid Examples	**Invalid Examples**
nums=[1,2,3];	mixed=[1,'two',3]; (mixed element types)
words=["str",'strs'];	trail=[1,2,]; (trailing comma)
computed=[1+1, square(2)];	unclosed=[1,2; (missing closing bracket)



grid=[[1,2],[3,4]];	box=[[1,2],["a"]]; (nested lis mismatch)

A.5.3 Indexing

1. Elements are accessed with zero-based subscripts <**lis**>[<index>], where the index must be a **wharfa** value (literal or expression).
    a. A **non-wharfa** index is an error.
    b. The valid range is 0 to length−1; any other **wharfa**, including negatives, is an out-of-range runtime error and crashes.
2. A subscript denotes a storage location as well as a value:
    a. Read: <lis>[<i>] evaluates to a value of the element type anywhere such a value is expected.
    b. Write (index assignment): <lis>[<i>] = <expr>; replaces element i in place. RHS must match the element type (compile-time error otherwise) and length is unchanged.
    c. Augmented write: <lis>[<i>] += | -= | = | /= | %= reads the element, applies the operator per B/H and stores back. The result must match element type. %= is wharfa-only.
    d. Rule 1's index rules apply identically to writes. An out-of-range write is a runtime crash.
    e. A write whose chain roots at a const name is an error.
3. Chained subscripts index nested lis left to right on reads and writes:
    a. grid[i][j]=v; updates one innermost element. Meanwhile, grid[i]=<lis value>; replaces one row, which must match the row element type.
4. Only length and element type are immutable.
    a. Elements cannot be added, removed, or shifted after creation.
    b. Element values are editable in place.
    c. The only whole-value changes are reassignment and deleo.
5. insid and lood read elements and do not modify them by themselves.
    a. An index assignment in a loop body edits the iterated value and later iterations see the update.
    b. Assigning to the loop variable does not write back to the iterable.

Syntax
<identifier>[<wharfa expr>];
<identifier>[<i>][<j>];
<identifier>[<wharfa expr>] = <expr>;



<identifier>[<wharfa expr>] <augop> <expr>;
<identifier>[<i>][<j>] = <expr>;

Valid Examples	Invalid Examples
first=nums[0];	x=nums[3]; (out-of-range index)
cell=grid[0][1];	y=nums[1.5]; (non-wharfa index)
belial(nums[1]==2){ prin("two"); }	z=nums["0"]; (non-wharfa index)
val=nums[0];	w=empty[0]; (empty out-of-range)
sum=nums[0]+1;	n=nums[-1]; (negative out-of-range)
w=nums[0];	nums[0]="a"; (element-type mismatch)
n=nums[2];	nums[5]=1; (out-of-range write)
nums[0]=9;	nums["0"]=1; (non-wharfa index on write)
nums[0]+=1;	nums[0]+=1.5; (promotion across store)

A.5.4 Nested lis

1. A **lis** whose elements are all **lis** is a three-dimensional **lis** (its elements are rows). Each further level of nesting adds one dimension. The dimensionality equals the depth of bracket nesting in the literal.
    a. A lis can have a maximum of 3 dimensions. Anymore is a semantic error.
    b. Mixing **lis** and **non-lis** elements at one level is an error.
    c. All inner **lis** at the same level must share one element type.
    d. Rows may differ in length; rectangularity is not required, because length belongs to each row value.
2. Initialization writes one pair of square brackets per dimension, outermost first.
3. The number of bracket pairs in the literal equals the number of dimensions; the number of subscripts in a full element access equals the same number.
4. Access uses one subscript per dimension, left to right:
    a. grid[i] yields the row at index i, a **lis** value. grid[i][j] yields the element.
    b. Fewer subscripts than dimensions yields an inner **lis**.



i. More subscripts than the value's dimensions is an error because some level subscripts a non-collection value.

c. A subscript applies only to a **lis** (**wharfa** index) or a **glossare** (**strina** key).

d. Applying [] to **wharfa**, **flo**, **trua**, or **strina** is an error.

5. Length immutability and element editability hold at every level:
   a. grid[0][1]=9; updates one element in place. The right-hand side must match the innermost element type, checked at compile time when the chain roots at a declared lis variable.
      i. On the other hand, grid[0]=[7,8]; replaces one row and is valid when the new row is a lis of the same element type.
   b. No statement adds or removes rows or elements at any depth.
   c. Whole-value reassignment of the outermost variable with matching element type remains valid and may differ in length.
   d. Nested writes follow aliasing:
      i. When two names, or a parameter and its argument, link to one lis value, a write at any depth through one name is visible through the other. Only whole-value reassignment rebinds a single name.
   e. A nested write whose chain roots at a const name is an error at every level.
6. . (Remove After Checked)
7. The element type of a 2D **lis** is "**lis** of X"; == compares recursively and [1,2] **insid** grid tests row equality.
8. A sequence-form **lood** over a 2D **lis** yields rows. The loop variable must be a declared **lis** of the same element type. A nested **lood** visits the elements.
9. **prin**()/**formo**() render nested brackets recursively.
10. A parameter or return declared **lis** carries no dimension or element type in the header. Any **lis** may be passed or returned.
    a. Element-type and dimension expectations inside the body, and the element type of a variable declared from a **lis-returning call**, are fixed by the actual value at run time;
    b. a mismatch crashes as a value error, exactly as for the empty **lis**.

Syntax
<identifier> = [[<value>, <value>, ...], ...];
<identifier>[<i>][<j>];



Valid Examples	Invalid Examples
grid=[[1,2],[3,4]];	mixed=[[1,2],3]; (lis and wharfa at one level)
ragged=[[1,2,3],[4]];	badrow=[[1,2],["a"]]; (inner element types differ)
cell= grid[0][1];	deep= grid[0][0][0][0]; (subscripts exceed dimensions)
row= grid[1]; first= row[0];	grid[0]=["a"]; (element-type mismatch)
rv=[0,0]; lood(rv insid grid){ prin(rv[0]); }	t= grid[0]["a"]; (non-wharfa index)
belial([1,2] insid grid){ prin("row"); }	tr= [[1,2],]; (trailing comma)
prin(grid);	s= grid[5][0]; (out-of-range row)
grid[0][1]= 9;	exceed= [[[[1]]]]; (4 Dimensions)
grid[0]=[7,8];	

A.6 Glossare

glossare is the key-value mapping type of Pothos:

1. A glossare literal must contain at least one entry.
    a. An empty literal {} results in a syntax error.
    b. Like lis ([]), glossare ({}) does not permit empty literals. Only strina ("") may be empty.
    c. A function with no entries to return must use a sentinel entry or another type.
    d. The lexer recognizes { and } as separate valid tokens.
        i. An empty code block {}, such as elsa {}, is permitted and does not produce an error.
2. Keys must be unique strina literals.
    a. A duplicate key is an error.
    b. Keys may be enclosed in either single (') or double ("") quotation marks.
        i. "a" and 'a' represent the same key. Using both in the same glossare literal, however, results in a duplicate key error.
    c. An empty strina literal ("") or (') is permitted as a key, provided it is unique.
        i. For example, {"": 1} is valid, while {"": 1, ': 2} produces a duplicate key error.



    d. Keys are case-sensitive. Uppercase and lowercase letters are treated as distinct characters.
        i. For example, "A" and "a" are different keys and may coexist in the same glossare literal.

3. Values may be any valid expression; each is evaluated once, in entry order, at creation time, and the stored value is the resulting value.
    a. A value may result in any of the six data types (**strina**, **wharfa**, **flo**, **trua**, **lis**, **glossare**), and one **glossare** may mix value types: {"a": 1, "b": "x"} is valid.
        i. Unlike **lis**, **glossare** has no homogeneity requirement.
    b. **lis** and **glossare** are accepted values (nested collections).
    c. Later changes to variables used in a value expression do not affect the stored value.
4. Keys are static and literal-only.
    a. A variable or expression in the key slot is an error.
5. The key set is static after creation. Keys cannot be added, removed, or renamed, so the entry count is fixed, but values at existing keys are editable in place:
    a. Three write forms exist and must not be confused:
        i. Entry assignment at an existing key, which edits one value in place and may change that entry's stored type.
        ii. Key addition, a write whose key slot names a key outside the key set, which is an error.
        iii. And, whole-value reassignment of the variable, which replaces the entire mapping and is the only write that may change the key set.
    b. Entry assignment <glossare>["key"] = <expr>; replaces the value at an existing key. The key slot takes a quoted strina literal as on reads. And, the new value may be any of the six types, so replacement may change the entry's stored type.
        i. Every later use of that entry re-checks its type at run time.
        ii. Entry assignment never adds keys.
    c. Writing a key outside the key set is key addition and an error, same stage as reading a missing key (run time).
        i. Key addition is distinct both from entry assignment, which targets an existing key, and from whole-value reassignment, which replaces the key set itself.
        ii. No statement inserts a key into an existing glossare value.
    d. Entry augmented assignment follows H (read→operate→store), operand types checked at run time against the entry's current value.



e. Chained subscripts write as they read, left to right: store["inner"]["a"]= 2; replaces one innermost entry and store["nums"][0]= 9; replaces one element of a lis reached through a glossare.
   i. Each subscript level follows the rules of the collection it reaches: entry replacement may store any of the six types, while a level reaching a lis element must match that lis's element type.
   ii. Levels reached through a glossare entry are checked at run time.
f. Whole-value reassignment (<glossare> = <glossare value>;) remains the only whole-mapping change:
   i. it links the variable to a brand-new glossare value whose key set may differ from the old one, while the variable's inferred type stays glossare.
   ii. It edits nothing in place, and any other name linked to the old value keeps it (aliasing).
g. Entry writes rooted at a const name are errors.

6. Values are accessed using square brackets: dict["key"]. The same subscript form appears on the left of = for entry assignment and on the left of an augmented operator for entry augmented assignment.
   a. Accessing a missing key is an error.
   b. Calling a key without quotes is an error.

Syntax
<identifier> = {<strina_key>: <value>, <strina_key>: <value> …};
<identifier>[<strina key>] = <expr>;

Valid Examples	**Invalid Examples**
user={"name": "Ana", "score": 95};	x= user["age"]; (missing key)
name= user["name"];	dup={"a": 1, "a": 2}; (duplicate key)
mix={"a": 1, "b": "x"};	v= user[random]; (undeclared key)
store={"nums": [1,2], "inner": {"a": 1}};	emptyDict={}; (empty glossare)
first= store["nums"][0]; deep= store["inner"]["a"];	user["age"]+=1; (missing key on read)
base= 1; snap={"n": base, "sum": base+ 1}; base= 9;	user["age"]= 30; (key addition; key set static)



user["name"]= "Bob";	k="a"; d={k: 1}; (non-literal key)
store["inner"]["a"]= 2; store["nums"][0]= 9;	d={"a": 1,}; (trailing comma)
user["score"]="A"; user={"name":"Bob","age":30};	store["missing"][0]= 1; (missing key)
	store["nums"][0]="a"; (element-type mismatch)
	user+= {"age":30}; (no augmented operator)

B. Operators

Operators are symbols and reserved words that combine, compare, or test values to produce a new value. Operand type requirements are semantic checks applied after parsing: the grammar parses every expression shape without regard to types, and each operator then verifies its operand types at the earliest stage at which both are known. Compile time: literals, declared identifiers (whose inferred type is immutable), parameters, and calls to user-defined or built-in functions (whose return type is declared). Run time: operands whose type is fixed only by the produced value, namely lis elements of a run-time-typed lis and glossare entry values. Both operands are evaluated first, left to right, and the operator's type rule is then applied. A violation is an error at that stage. Promotion is part of this check: a wharfa/flo pair passes, every other cross-type pair fails.

B.1 Arithmetic Operators:

1. The '+'operator adds two **wharfa**, two **flo** values, or a **wharfa** value with a flo value, or concatenates two **strina** values.
    a. Mixing a strina operand with a wharfa or flo operand is a type error:
        i. Compile time when both operands are statically typed and likewise for identifiers and calls.
        ii. Run time when one operand's type is fixed only at run time, as in a glossare entry holding a strina added to a wharfa.
2. '-' returns the difference of two **wharfa** or **flo** values; the same symbol also serves as unary negation for negative numbers.
    a. The parser reads '-' as unary negation when it appears in operand position: At the start of an expression, or immediately after '(', ',', '=', an augmented assignment operator, a relational operator, 'reor', '[', '{', ':', '+', '*', '/', '%', or another unary '-'.



   b. The parser reads '-' as binary subtraction when it appears in operator position: immediately after an operand (a literal, an identifier, ')', or ']').
   c. Unary negation applies only to **wharfa** or **flo** operands; negating any other type is an error.
   d. Spacing does not affect the choice:
      i. x = -5; and x = - 5; are both negation;
      ii. x = 5 - 3; and x = 5 -3; are both subtraction.
   e. A '-' immediately followed by another '-' is lexed as the single-line comment start '--'.
      i. To negate twice, write '- -5' with a space.
3. '*' returns the product of two **wharfa** or **flo** values.
4. '/' returns the integer quotient of two **wharfa** values, discarding decimals, or the decimal quotient of two **flo** values, or of a **wharfa** and a **flo** value, rounded half-up to 4 fractional digits.
   a. Integer / drops the decimals and rounds toward zero (e.g: -<value> / <value> = -<value>).
5. '%' returns the remainder of two **wharfa** values only.
   a. A **flo** operand or a mixed pair with '%' is a type error; '%' does not promote.
   b. % keeps the sign of the left side (e.g: -<value> % <value> = -<value>).
6. Both operands of an arithmetic operation must be numeric (**wharfa** or **flo**). Same-type pairs operate in their own type, and mixed pairs operate under the next rule.
   a. A non-numeric operand is an error: compile time for statically typed operands (literals, identifiers, parameters, calls with declared return types), run time for operands whose type is fixed only at run time (lis elements of a run-time-typed lis, glossare entry values).
   b. Division by zero is an error.
      i. This applies to both '/' and '%': if the right operand evaluates to zero, it is a runtime error and the program crashes immediately.
      ii. The check occurs at runtime, after both operands are evaluated; a zero arising from **innum**() or from a computation is likewise an error.
      iii. No silent substitute result (such as 0) is ever produced, leading to an error.
7. Mixed **wharfa**/**flo** promotion: when one operand is **wharfa** and the other is **flo** in '+', '-', '*', '/', or any relational operator, the **wharfa** operand is first converted to its exact **flo** equivalent (fixed point, 4 fractional digits); the operation then proceeds as **flo** and produces a **flo** value.



a. The conversion is exact: every **wharfa** value in the allowed range maps to a representable **flo** value.
b. Promotion applies only inside operator evaluation. It does not apply to assignment, augmented assignment, argument passing, return values, lis element homogeneity, **insid** operands, or **lood** element checks.
c. '%' never promotes: both operands must be **wharfa**.

Syntax
<wharfa | flo> + <wharfa | flo>
<wharfa | flo> - <wharfa | flo>
- <wharfa | flo> (unary negation)
<wharfa | flo> * <wharfa | flo>
<wharfa | flo> / <wharfa | flo>
<wharfa> % <wharfa>

Valid Examples	**Invalid Examples**
x = 7 / 2;	b = 5 / 0; (division by zero)
y = 7 % 2;	g = 5 % 0; (modulo by zero)
n = -5; m = - 5;	q = -"a"; (negation of non-numeric)
d = 5 - 3;	m = 5% 2.0; (mixed modulo)
p = 7.0 / 2.0;	s = 5.0 % 2.0; (modulo not defined for flo)
t = 1 + 1.5;	

B.2 String Concatenation:

1. '+' joins two **strina** values left to right and produces a **strina**.
2. Concatenating a **strina** with any other data type is an error; no implicit conversion exists.

Syntax



<strina> + <strina>

Valid Examples	Invalid Examples
z = "Po" + "thos";	c = "n=" + 5; (concatenation with integer)
full = "Ana" + " Cruz";	a = 5 + "a"; (type mismatch)
chained = "a" + "b" + "c";	v = "s" - "t"; (subtraction of strina)

B.3 Relational Operators:

1. The relational operators '==', '!=', '<', '<=', '>', '>=' compare two values of the same type, or a **wharfa** value with a **flo** value under promotion, and produce a **trua** value.
2. '<', '<=', '>', '>=' order **wharfa** or **flo** values, including mixed pairs; '==' and '!=' test equality of any two values of the same type, or of a **wharfa** and a **flo** (promoted).
    a. Comparing values of different data types, other than the **wharfa**/**flo** numeric pair, is an error.
3. Two **lis** values are equal under == when they have the same length and every pair of corresponding elements is equal; equality recurses into nested **lis**.
    a. A length or element mismatch yields **Inaccu** for == (**Trua** for !=), not an error.
    b. Comparing **lis** of different element types is an error.
    c. Two **glossare** values are equal when they have the same key set and equal values at every key.

Syntax
<same-type> == <same-type>
<same-type> != <same-type>
<wharfa | flo> < | <= | > | >= <wharfa | flo>

Valid Examples	Invalid Examples
result = 3 > 2;	d = 3 > "a"; (different types compared)
eq = "a" == "a";	r = [1] < [2]; (ordering defined only for wharfa/flo)



ne = 1 != 2;	[1,2] == ["a","b"] (different element-types)
ord = 1.5 <= 2.5;	[[1,2]] == [["a","b"]]; (nested type-mismatch)
result_true = [1, 2] == [1, 2];	[1,2] == [[1,2]]; (mismatched element-types)
result_false = [1, 2] == [1, 2, 3];	
w = 1 == 1.0;	
ord2 = 2> 1.5;	

B.4 Logical Operators:

1. The logical operators are the reserved words '**an**' (and), '**othan**' (or), and '**neve**' (not), and require **trua** operands.
    a. A **non-trua** operand is an error.
2. '**an**' produces **Trua** only when both operands are **Trua**.
3. '**othan**' produces **Trua** when at least one operand is **Trua**.
4. '**neve**' negates a single **trua** value.
5. Logical expressions produce a **trua** result.

Syntax
<trua> an <trua>
<trua> othan <trua>
neve <trua>

Valid Examples	**Invalid Examples**
active = Trua; ready = Trua; both = active an ready;	e = 5 an 3; (non-trua operands)
either = active othan Inaccu;	o = Trua othan 1; (non-trua right operand)
neg = neve active;	nv = neve "x"; (negation of non-trua)

B.5 Membership Operator:

1. '<value> **insid** <collection>' tests whether a value is contained in a **lis** or **strina**.
    a. The expression produces **Trua** if the value is found, otherwise **Inaccu**.



2. For a **lis**, the value is compared against each element; for a **strina**, the left operand must be a **strina** contained within the right operand.
    a. Using **insid** with unsupported operand types is an error.
3. '**insid**' also serves as the iteration keyword of sequence-form **lood** headers, membership testing on a **glossare** is not supported.
4. **insid** is non-associative: a insid b insid c is a syntax error.

Syntax
<value> insid <lis | strina>

Valid Examples	**Invalid Examples**
found = 2 insid [1, 2, 3];	f = 2 insid "hello"; (unsupported operands)
sub = "lo" insid "hello";	h = "name" insid user; (glossare membership unsupported)
missing = "z" insid "hello";	i = [1] insid [1, 2]; (left operand type differs from elements)

C. Identifiers

Identifiers are the case-sensitive names given to variables and functions; reserved words are the fixed vocabulary of the language with predefined meanings:

1. An identifier must begin with a letter (a-z, A-Z).
    a. Beginning with a digit or underscore is an error.
2. Succeeding characters may be letters, digits (0-9), or underscores (_) only.
    a. Spaces and special symbols are an error.
3. An identifier has a maximum length of 20 characters.
    a. A minimum of 1 character for variables and 3 characters for functions.
    b. Exceeding 20 characters is a lexical error.
        i. The lexer will accept the first 20 characters and will process the excess as separate tokens.
    c. A function name shorter than three characters is a syntax error, reported by the parser at the function declaration. The lexer accepts such a name as an identifier.
        i. A call such as ab(1) fails as an undeclared function.
4. Global identifiers are case-sensitive and unique among active identifiers:



   a. At any moment no two live objects (variable, constant, function) share a name, and no scope shadows another.
   b. A global variable, constant, or function may be declared only once per lifetime; the name of a deleted non-constant variable leaves the symbol table and may be declared again later, which begins a new lifetime.
   c. Redeclaring an identifier that is currently active is an error, whether the second declaration appears at top level or inside a function.
      i. Simultaneous collisions and shadowing are never authorized. Only a name freed by deleo (non-constant variables only) may be declared again.
   d. Function parameters are local identifiers. They must be unique within the parameter list of their function.
   e. A parameter may not duplicate the name of an active global variable, constant, or function, or of any reserved word.
      i. A name freed by deleo is no longer a global name and may serve as a parameter.
5. Reserved words cannot be used as identifiers.
6. Reserved words are written only in lowercase letters, except in the truth literals '**Trua**' and '**Inaccu**', which are capitalized and are case-sensitive.
   a. Writing a reserved word in any other casing (e.g, '**Belial**', '**inaccu**') is an error.

Valid Examples	**Invalid Examples**
A	2ndName (starts with a digit)
score	first name (contains a space)
userName	total$ (contains a special symbol)
total_amount	Belial (reserved word as identifier)
add	_age (starts with an underscore)
studentRegNumber2026	studentRegistrationNumber2026 (exceeds 20 characters)
omega = 1; smile = 5;	Ωmega = 1; 😀 = 5; (Non-ASCII characters)

temp= 1;
deleo temp;
temp= 2;
	temp= 1;
	const temp= 2;
	(active name)



	x= 1;
	wharfa foo(wharfa x){
	reor x;
	}
	(parameter duplicate)

D. Expressions

An expression is a combination of literals, identifiers, operators, and function calls that evaluates to a single value:

1. Every expression evaluates to a value of one of the 6 data types.
2. The value slot of an assignment, argument, condition, or return is fully evaluated before it is stored or used.
3. Parentheses '()' may group sub-expressions; grouped sub-expressions are evaluated first.
4. Every operand must satisfy the type requirements of its operator.
    a. A violation of an operator's type requirement is an error.
    b. These requirements are semantic.
        i. The grammar of this section parses any operand shape, and each operator's type rule is verified after parsing, at compile time when both operand types are statically known (literals, declared identifiers, parameters, calls with declared return types), otherwise at run time when the operand value's type is fixed (lis elements of a run-time-typed lis, glossare entry values).
    c. An identifier contributes its inferred, immutable type.
        i. A call contributes the callee's declared return type or a built-in's fixed return type. Therefore, combinations involving identifiers and calls are rejected at compile time, before any call executes, unless the operand reaches a run-time-typed element or entry.
5. An expression that cannot be evaluated is an error.

Precedence	Operator	Associativity
1	() grouping, [] indexing, function call	left
2	unary - (negation), neve (logical NOT) (unary -, neve)	right
3	*, /, %	left



4	+ (addition / concatenation), - (subtraction)	left
5	<, <=, >, >=	left
6	insid	non-associative
7	==, !=	left
8	an	left
9	othan	left

Valid Examples	**Invalid Examples**
grp = (2 + 3) * 4; (grouping first)	d = 5 an 3; (operator type violation)
neg = -3 * 2; (unary binds before *)	e = ghost + 1; (undeclared identifier)
mul = 7 / 2 * 3; (multiplicative, left→right → 9)	f = nums[9] + 1; (out-of-range index)
add = 1 + 2 - 3; (additive, left→right → 0)	g = 5 / 0; (division by zero)
rel = 1 + 1 > 1; (relational after additive)	i = 3 > "a"; (different types compared)
eq = 2 * 2 == 4; (equality after multiplicative)	j = 2 insid "hello"; (unsupported operands)
mem = 2 insid [1, 2, 3] == Trua; (insid binds before ==)	k = (2 + 3; (unbalanced parenthesis)
conj = Trua an 2 > 1; (relational binds before an)	l = neve 5; (neve on non-trua)
disj = Inaccu othan 1 == 1; (== binds before othan)	
h = 1 + 1.5;	

E. Statements and Code Blocks

A statement is a group of reserved words, identifiers, literals, operators, and functions working together to perform one action; a code block groups statements into a single executable unit:



1. Simple statements must end with ';'. Function definitions, conditional, loop, and branch blocks all end with the closing brace '}'.
    a. A missing semicolon is an error.
    b. A ; after the } of belial, belsa, elsa, cycla, lood, or a function declaration is a syntax error.
        i. Empty statements are not allowed.
2. Statements generally start with a reserved word, followed by an identifier, an operator, and a value; some statements may start with an identifier instead.
    a. Simple statements include:
        i. assignment statement
        ii. augmented assignment statement
        iii. **const** declaration statement
        iv. **trua** <identifier>; declaration statement
        v. function call statement
        vi. **reor** return statement
        vii. **paus** break statement
        viii. **persis** continue statement
        ix. **pas** no-op statement
        x. **deleo** removal statement
    b. An expression standing alone is not a statement. An expression becomes a statement only inside the forms listed above.
        i. The function call is the one expression form that is itself a simple statement when written alone.
    c. Every simple statement ends with ;, including a function call used as a statement, whose value is discarded after evaluation.
        i. i. Control-flow statements(belial, belsa, elsa, cycla, lood) end with their closing brace and take no semicolon.
        ii. A function declaration ends with }. A semicolon never turns a bare expression into a statement.
3. Anything that returns a value is valid inside the value slot of a statement.
4. A code block is a logical grouping of statements delimited by curly braces '{ }'.
5. A code block executes exclusively under three conditions: when activated by a conditional statement, when activated by a looping statement, or when invoked through a function call.
    a. A bare code block standing alone as a statement is a syntax error.
6. The standalone statement '**pas**;' does nothing.
7. Unbalanced curly or square braces or parentheses are syntax errors.
8. A comma immediately before a closing], }, or) is an error.



Syntax
<statement>;
{ <statement/s> }

Valid Examples	Invalid Examples
prin("Hello");	prin("Hello") (missing semicolon)
x = 1; pas;	{ prin("Hi"); (unbalanced brace)
belial (x == 1) { prin(x); }	prin("Hi"; (unbalanced parenthesis)
foo(5);	{ x = 5; } (bare block, no header)
	5; [1,2] == [1,2]; (bare expression)

F. Comments

Comments are annotations that describe the code but are ignored by the compiler during execution:

1. A single-line comment begins with '--' only when it is not immediately followed by /, and runs to the end of the line.
    a. It ends at the next newline character or at the end of the file, whichever comes first
2. A multi-line comment begins with '/--' and ends with '--/'.
    a. A multi-line comment without a closing '--/' is an error (unterminated comment).
    b. A closing '--/' without an opening '/--' is an error.
    c. Nested multi-line comments are not supported. The first --/ closes the comment.
3. Comments may contain ASCII letters, numbers, and special characters, and may appear anywhere in the program.
4. Comments are ignored by the compiler and produce no value.
    a. They do not count as definitions or statements, so a file containing only comments and whitespace is a syntax error.

Syntax



-- <comment>
/-- <comment> --/

Valid Examples	Invalid Examples
-- single line comment	/-- missing closer (unterminated comment)
/-- multi line comment --/	/-- a /-- b --/ c --/ (stray --/)
x = 5; -- trailing comment	--/ (missing opening /--)
	-- only comments in the file; (comments-only program)

G. Variables and Constants

Variables and constants are named storage locations that link an identifier to a value:

1. Variables and Constants are declared by pairing an identifier with a value using an assignment operator.
2. The variable's data type is automatically inferred from its assigned value at declaration and is immutable.
    a. Assigning a value of a different type afterwards is an error; variables cannot be recast.
    b. The only declaration form using a type keyword is 'trua <identifier>;' typed declarations are syntax errors.
3. The value slot may hold any literal, expression, function call, or variable, provided it guarantees a value matching the variable's type.
4. Variables must be declared before their values are accessed.
    a. The first assignment to an undeclared identifier implicitly declares it within the current scope, provided it does not refer to an existing variable in an accessible scope.
    b. Subsequent assignments reassign the variable's value without changing its original data type.
    c. Accessing an identifier before its declaration or first assignment is an error.
5. A constant is declared with the '**const**' keyword and must be initialized at declaration.



a. Reassigning a constant is a semantic error.
b. Incrementing/decrementing a constant is a semantic error.
c. A constant's linked value is deeply immutable: any assignment or augmented assignment whose target is a subscript rooted at the constant is an error.
d. Deleting a constant with **deleo** is a semantic error.

6. A constant's data type is automatically inferred from its assigned value at declaration and is immutable.
   a. Explicit data type declarations for constants are syntax errors.
7. Global variable names, constant names, and function names are unique program-wide among active identifiers: while an object lives, no second object may take its name at top level or in any function body.
   a. Function parameter names are local to the function in which they are declared.
   b. A deleted non-constant identifier may be declared again.
   c. A redeclaration after deleo is a fresh declaration: the new variable's type is inferred from its new initial value, and the deleted object's value and type are not restored.

Syntax
<identifier> = <value>;
const <identifier> = <value>;
trua <identifier>;

Valid Examples	**Invalid Examples**
age = 19;	age = 19; age = "nineteen"; (type change)
name = "Ana";	const LIMIT; (uninitialized constant)
const LIMIT = 100;	x = y + 1; (y undeclared)
total = age + 1;	LIMIT = 200; (constant reassignment)
	wharfa x= 5; (typed declaration)
	const K=[1,2]; K[0]=9; (constant element modification)



H. Assignment and Augmented Assignment

Assignment stores an evaluated value into a variable or into a subscripted element of a lis or glossare. Augmented assignment combines an arithmetic operation with assignment in one statement:

1. The assignment operator is '='; the right-hand side is evaluated first before being stored in memory. The target of = and of the augmented operators is an identifier or a subscript chain rooted at a declared, non-const lis or glossare variable.
2. The augmented operators are '+=', '-=', '*=', '/=', and '%='.
    a. 'x += 1;' is equivalent to 'x = x + 1;'
    b. Augmented string concatenation 'x += 'world';' is permitted.
    c. %= applies only to **wharfa** variables
3. The resulting value must match the existing type of the variable.
    a. for a **lis** variable, matching means the same element type; the new value's length may differ.
    b. Promotion never crosses the assignment: an augmented operation whose promoted result is **flo** cannot be stored into a **wharfa** variable, and vice versa.
    c. For a store target, the result must match the location's existing type:
        i. The element type for a lis element store, and the entry's current value type for a glossare augmented store, are checked at run time.
        ii. A plain glossare entry assignment is exempt from same-type matching and may store any of the six types, after which the stored type becomes the entry's current type.
        iii. Promotion never crosses the store.
4. Augmented assignment may only be applied to an already declared variable or to a subscript of one.
    a. Applying it to an undeclared identifier is an error.
5. Trying to rewrite a constant using augmented operators is an error.

Syntax
<store_target> <assops> <value>; (where <assops> is '=', '+=', '-=', '*=', '/=', '%=')
<store_target>=<identifier> | <store_target>[<wharfa expr>](lis subscript) | <store_target>[<strina key>](glossare subscript)

Valid Examples	**Invalid Examples**



a = 5; b = 2;
c = 10; d = 3;
e = 8; f = 5;
	g -= 1; (g undeclared)

b += 3; c -= 2; d*= 4;	s = "X"; s -= 1; (type-incompatible operation)
e /= 2; f %= 3;	const K = 1; K += 1; (constant modification)
x += 'world';	i = 1; i+= 1.5; (promoted wharfa)

nums[0]=9;
nums[0]+=1;
user["name"]="Bob";	

I. Built-In Functions

Built-in functions are always available without declaration, and their names are reserved:

I.1 prin()

1. '**prin**()' prints values to standard output.
2. Arguments may be literals, variables, expressions, or function calls.
   a. prin takes strictly one value and may use formo() to join values.
   b. prin() is a syntax error.
   c. prin always ends with a newline
3. String arguments must be quoted **strina** literals (double or single quotes).
4. The arguments must be enclosed in parentheses, and the statement must end with a semicolon ';' just outside them.
   a. Missing parentheses or semicolon is an error.

Syntax
prin(<expression>);

Valid Examples	**Invalid Examples**
prin("Hello, World!");	prin"Hello"; (missing parentheses)
score = 90; prin(score);	prin(score; (unbalanced parenthesis)
prin(a + b); prin (1 + 1);	prin(Hello); (undeclared identifier/unquoted string literal)



I.2 inpub()

1. '**inpub**()' reads one line of user input from the console.
2. **inpub**() always returns a **strina** value.
3. The returned value may be assigned to a variable. The variable's inferred type is **strina**.
4. **inpub**() must be called with empty parentheses and exact casing.
    a. **inpub**(or **Inpub**() is an error.

Syntax
<identifier> = inpub();

Valid Examples	**Invalid Examples**
name = inpub();	Parenthesis = inpub(; (unbalanced parenthesis)
capitalization = inpub();	capitalization = Inpub(); (wrong casing)

I.3 innum()

1. '**innum**()' reads one line of user input from the console.
2. **innum**() always returns a **wharfa** value.
3. The input must be a valid **wharfa** integer.
    a. Spaces are trimmed and '-' is optional.
    b. Digits are allowed only, anything else is a run-time error.
    c. If the value is outside the range -99999999 to 99999999, it is a runtime error.
4. **innum**() must be called with empty parentheses and exact casing.
    a. **innum**(or **Innum**() is an error.

Syntax
<identifier> = innum();

Valid Examples	**Invalid Examples**
age = innum();	x = innum(; (unbalanced parenthesis)
score = innum();	y = Innum(); (wrong casing)



prin(age + 1);	z = innum(); (strina input)

I.4 inflo()

1. **inflo**() reads one line of user input from the console.
2. **inflo**() always returns a **flo** value.
3. The input must be a valid **flo** literal: an optional leading -, then at least one digit on both sides of a single decimal point; integer part capped at 8 significant digits, fractional part at 4 digits.
    a. Non-numeric characters, a missing digit on either side of the point, or a second point are runtime errors.
    b. Input exceeding the integer or fractional caps is a runtime error.
    c. A leading + is a runtime error.
    d. Integer-shaped input (e.g. 5) is a runtime error: each reader accepts precisely the literal shape of its return type.
    e. Leading zeros in the integer part are discarded; the value is stored as fixed-point with 4 fractional digits.
4. The returned value may be assigned to a variable; the variable's inferred type is **flo**.
5. **inflo**() must be called with empty parentheses and exact casing.
    a. **inflo**(or **Inflo**() is an error.

Syntax
<identifier> = inflo();

Valid Examples	**Invalid Examples**
price = inflo();	p = inflo(; (unbalanced parentheses)
rate = inflo();	P = Inflo(); (wrong casing)
t = inflo(); prin(t + 1.0);	input: 5, .5, 5., 1.23456, +2.5 (runtime errors)

I.5 formo()

1. '**formo**()' formats a **strina** by substituting placeholders with the values of variables.
2. **formo**() must be called with parentheses enclosing a quoted **strina** literal.
    a. Writing **formo** without parentheses is an error.



3. } Placeholders are written{<expression>} inside the format string. Its resulting value is interpolated. A literal or an identifier alone is also an expression. The format itself must be a strina literal.
    a. Every identifier used inside a placeholder expression must be declared before the formo() call executes.
        i. A placeholder expression using an undeclared identifier is an error.
    b. Only curly braces interpolate: '( )' and '[ ]' around a name remain a literal text. Placeholder braces are interpolation markers inside the format string. They are neither glossare-literal braces in an expression nor code-block braces at statement level.
    c. No brace-doubling escape exists. A literal{ or} in the output is produced by interpolating a strina literal, as in {'{'} or {'}'}.
    d. : A placeholder holds one expression. Spaces around and within a valid expression are permitted.
        i. A placeholder whose expression yields no value is an error, and an assignment or augmented assignment is never a valid placeholder expression.
    e. A lone { or } is a semantic error.
    f. Placeholder scanning runs on the resolved value (after escape processing), so \{name} is a lexical error.
        i. Braces nested inside a placeholder expression (as in a glossare literal) are balanced by the scanner, and the placeholder ends at the matching unbalanced closing brace.
    g. Quotes inside the format value open/close nothing during interpolation; substituted values are inserted as is.
    h. A strina literal inside a placeholder expression must use the quoting style opposite to the format string's delimiter:
        i. formo("...{'x'}...") is valid, while formo("...{"x"}...") ends the literal at the inner" → lexical error.
4. **formo**() returns a **strina** and may appear on the right-hand side of an assignment, as an argument to **prin**(), or anywhere a **strina** value is expected.

Syntax
formo("<text{expression} text>");

Valid Examples	**Invalid Examples**
varx = "Sean"; vary = 19;	greet = formo"I am {varx}."; (missing



greet = formo("I am {varx}, age {vary}.");	parentheses)
varz="Charles"; hello= formo("I am{varz}.{'{'}literal{'}'}");	greet = formo("I am {varx}, age {vary}.; (unterminated string)
prin(formo("I am {varx}, age {vary}."));	greet = formo("I am {nobody}."); (undeclared placeholder)

formo("value={2}, sum={(x + 1)}");
formo("wide={ x + 1 }");
formo("brace={'{'}");
	formo("v={x +}"); (placeholder expression yields no value)

q = 'She said "hi"'; prin(formo("q={q}"));	formo("a={x = 1}"); (assignment inside a placeholder)
	formo("\{name}"); (invalid escape sequence)

I.6 Display Forms

1. **prin**() and **formo**() render values by type:
    a. **wharfa**: normalized digits (leading zeros discarded), - prefix when negative.
    b. **flo**: 4 fractional digits, trailing zeros trimmed, at least one kept.
    c. **trua**: **Trua** / **Inaccu**.
    d. **Strina** (standalone argument): raw characters with escapes resolved, no surrounding quotes.
    e. **lis**: elements rendered by these rules, separated by ', ', enclosed in []; **strina** elements inside a collection appear in double quotes with internal escapes (\", \\, ...).
    f. **glossare**: entries as "key": value in key-insertion order, separated by ', ', enclosed in {}.
    g. Nested collections render recursively under the same rules.
2. A formo() placeholder interpolates the rendering prin() would produce for the value of its expression (so a strina value interpolates raw, but a strina element inside an interpolated lis keeps its quotes).
3. Display forms are output text only; they are not values or expressions in the program.

Input	Output
prin(100);	100
prin(3.50);	3.5



is_true = Trua; prin(is_true);	Trua
prin("hello");	hello
prin([1,2,3]);	[1, 2, 3]
prin([[1,2],[3,4]]);	[[1, 2], [3, 4]]
prin({"n":1,"s":"x"});	[]
nums = [1, 2]; prin(formo("v={nums}"));	{"n": 1, "s": "x"}
	v=[1, 2]

J. Conditional Statements

Conditional statements allow the program to choose which code block to execute based on a condition:

1. A conditional statement begins with '**belial**'.
2. The condition must be enclosed in parentheses.
3. The condition must evaluate to a '**trua**' value.
    a. A condition of any other type is an error.
4. The body must be enclosed in curly braces '{ }'.
5. Additional branches may be written using '**belsa**', each requiring its own condition in parentheses.
6. A final optional fallback branch uses '**elsa**', which takes no condition.
    a. Giving '**elsa**' a condition, or placing any branch after '**elsa**', is an error.
7. Conditional blocks may be nested.
8. The semicolon rule applies only to simple statements written inside the block (assignment, augmented assignment, const declaration, **trua** declaration, function call, **reor**, **paus**, **persis**, **pas**, **deleo**); each of these must end with a semicolon ';'.
    a. Conditional statements (belial/belsa/elsa) and looping statements(cycla/lood) are control-flow statements; therefore they take no trailing semicolon. The closing brace'}' alone ends the statement.
    b. A ' ; ' placed immediately after the ' } ' of a belial, belsa, elsa, cycla, lood, or function declaration is a syntax error.
        i. Empty statements are prohibited.



Syntax

belial (<condition>) {
<statement/s>
} belsa (<condition>) {
<statement/s>
} belsa (<condition>) {
<statement/s>
} elsa {
<statement/s>
}

Valid Examples	Invalid Examples
score = 75; belial (score >= 90) { prin("Excellent"); } belsa (score >= 75) { prin("Good"); } elsa { prin("Needs improvement"); }	belial score > 70 { prin("Pass"); } (missing parentheses)

inventory = ["apple", "banana", "cherry"];
target = "banana";
 	
belial (target insid inventory) {
prin("Item is in stock.");
}
	belial (score > 70) prin ("Pass"); (missing braces)

role = "admin";
active = Trua;
belial (active an role == "admin") {
prin("Admin panel.");
} belsa (active an role == "editor") {
prin("Editor tools.");
} elsa {
prin("Read only.");
}
	belial (score) { prin("Pass"); } (condition not trua)

age = 16;
has_id = Trua;
belial (age >= 18) {
belial (has_id) {
prin("Entry allowed.");
} elsa {
prin("ID required.");
}
} elsa {
	elsa (score >= 74) { prin("Fail"); } (elsa condition)



prin("Students only."); }	

K. Looping Statements

Loops repeat a code block, either while a condition holds '**cycla**' or once per element of an iterable '**lood**':

K.1 Cycla Loop

1. A '**cycla**' loop begins with the reserved word '**cycla**'.
2. The condition must be enclosed in parentheses and must evaluate to a '**trua**' value.
   a. A **non-trua** condition is an error.
3. The loop body must be enclosed in curly braces '{ }'.
4. The condition is checked before each iteration.
5. The loop continues while the condition is '**Trua**', and stops when it becomes '**Inaccu**'.
6. The loop body may contain any valid statement, including:
   a. assignment, augmented assignment, **const**, and **trua** declaration statements;
   b. function call statements (**prin**, **inpub**, **formo, inflo**, **innum**, or **user-defined**);
   c. conditional statements (**belial**/**belsa**/**elsa**), which may nest inside the loop;
   d. other looping statements (**cycla** or **lood**); loops may be nested;
   e. loop control statements (**paus**, **persis**), which act on the nearest enclosing loop (L);
   f. the no-op statement **pas**;.
7. A bare condition is not a statement; a condition may appear only inside the parentheses of a **belial**, **belsa**, **cycla**, or **lood** header.
8. **reor** may appear inside a loop body only when the loop itself is inside a function body.
9. Pothos has no increment (++) or decrement (--) operators. These operators must instead be written as i += 1; for increment, and i -= 1; for decrement.

Syntax
cycla (<condition>) { <statement/s> }

Valid Examples	**Invalid Examples**
count = 5; cycla (count > 0) { prin(count); count -= 1; }	cycla count > 0 { count -= 1; } (missing parentheses)



count = 3;
i = 0;
cycla (count > 0) {
  lood (i insid [1, 2]) {
  prin(i);
  }
  belial (count == 2) {
  paus;
  }
  count -= 1;
}
	cycla (count > 0) count -= 1; (missing braces)

K.2 Lood Loop

1. A '**lood**' loop begins with the reserved word '**lood**'.
2. . In the sequence form header, the left side of the reserved operator 'insid' is a previously declared loop variable (a bare identifier). Anything after insid may be any expression, including a direct lis or strina literal, provided it evaluates to a lis or strina value:
    a. a lis or strina literal (lood(i insid[1, 2]){}), a declared lis/strina variable, a call to a function whose return type is lis or strina, or any other expression that evaluates to a lis or strina. An expression that yields any other type is a valid expression but not a valid iterable.
    b. Anything other than a declared identifier on the left side of '**insid**' (a literal, an expression, or an undeclared name) is an error.
    c. An iterable expression that yields any type other than lis or strina (wharfa, flo, trua, or glossare) is an error. Only 'lis' and 'strina' are iterable.
3. The variable placed before '**insid**' must already be initialized (declared) before the loop.
    a. An uninitialized loop variable is an error.
    b. This requirement is applied only to the sequence form, while the counted form's init clause may declare the variable.
4. The loop header must be enclosed in parentheses, and the loop body in curly braces '{ }'.
5. The loop variable receives each element from the iterable in order, one per iteration.
    a. The iterable is evaluated once before the first iteration. The loop reads elements of that same value each iteration, so index assignments in the body are visible to later iterations. Assigning to the loop variable itself does not write back
6. The loop variable follows identifier rules.



7. The iterable value may be a'lis' or a 'strina' while the header slot itself accepts any expression that yields one.
    a. A '**lis**' yields its elements.
    b. A '**strina**' yields its characters as one-character '**strina**' values.
8. All iterated elements must be of the same data type as the loop variable.
    a. An element whose type differs from the loop variable's type is an error.
9. A **lood** loop also accepts a counted header: **lood**(<init>; <condition>; <step>){}.
    a. All three clauses are required; a missing clause or a missing/extra ';' delimiter is an error.
    b. The init clause is a simple assignment executed once before the first iteration; it declares the variable if undeclared (type inferred) or reassigns an existing same-type variable.
    c. The condition clause must evaluate to a **trua** value and is checked before each iteration.
    d. The step clause is an assignment or augmented assignment executed after each iteration of the body, including when **persis** is executed.
    e. **paus** exits the loop; **persis** skips to the step clause.
    f. A variable created by the init clause follows normal declaration rules and remains accessible after the loop, within the scope in which the loop appears (global at top level, local inside a function body).
    g. Pothos has no increment (++) or decrement (--) operators. These operators must instead be written as i += 1; for increment, and i -= 1; for decrement.
10. The loop body may contain any valid statement, including:
    a. assignment, augmented assignment, **const**, and **trua** declaration statements;
    b. function call statements (**prin**, **inpub**, **formo**, **innum, inflo**, or **user-defined**);
    c. conditional statements (**belial**/**belsa**/**elsa**), which may nest inside the loop;
    d. other looping statements (**cycla** or **lood**); loops may be nested;
    e. loop control statements (**paus**, **persis**): **paus** exits the nearest enclosing loop; **persis** skips to the step clause in counted form or to the next element in sequence form;
    f. the no-op statement **pas**;
11. A bare condition is not a statement; a condition may appear only inside the parentheses of a **belial**, **belsa**, **cycla**, or **lood** header, or in the condition clause of a counted header.
12. **reor** may appear inside a loop body only when the loop itself is inside a function body.
13. An empty iterable performs zero iterations; the loop variable keeps its previous value and no error occurs.



a. Because empty **lis** literals and empty **lis** values are prohibited everywhere, the only empty iterable that can occur in a program is the empty **strina** "": for example, **lood**(ch **insid** ""){} performs zero iterations.
b. A [] written in a **lood** header is therefore not a zero-iteration iterable but a syntax error (empty **lis** literal): **lood**(m **insid** []){} is rejected by the syntax analyzer, exactly as nums=[]; is.

Syntax

lood (<loopvar> insid <iterable_expression>) {
<statement/s>
}

lood (<init>; <condition>; <step>) {
<statement/s>
}

Valid Examples	**Invalid Examples**
nums = [1, 2, 3]; m = 0; lood (m insid nums) { prin(m); }	lood(n insid nums){ prin(n);} (uninitialized loop variable)
word = "hi"; ch = "h"; lood (ch insid word) { prin(ch); }	n= 0; lood n insid nums{ prin(n);} (missing parentheses around header)

sum = 0;
lood (i = 1; i <= 4; i += 1) {
sum += i;
}
prin(sum);
	m= 0; lood(m insid undeclaredList){} (undeclared iterable)

m= 0; lood(m insid [1, 2, 3]){ prin(m);}	lood(i=0; 5; i-=1){ } (condition is wharfa, not trua)
lood(i insid[1, 2]){ prin(i);}	m= 0; lood(m insid 2){}; (bare wharfa expression)
ch= "h"; lood(ch insid "hi"){ prin(ch);}	lood(j+=1; j<3; j+=1){ } (augmented assignment on undeclared identifier)
	lood(i=0; i<3){ } (missing step clause)
	lood(i=0; i<3; i++){ } (++ is not an operator in Pothos)



L. Loop Control Statements

Loop control statements alter the normal flow of the nearest enclosing loop:

1. '**paus**;' immediately exits the nearest enclosing loop.
    a. In nested loops, only the nearest loop is exited, and any outer loop continues.
2. '**persis**;' skips the remaining statements in the current iteration and proceeds to the next iteration.
    a. In counted lood form, persis transfers control to the step clause, which still executes; in sequence form, it advances directly to the next element.
3. '**paus**' and '**persis**' are valid only inside loops.
    a. Using either outside a loop body is an error.
4. Loop control statements must end with ';'.
5. '**paus**' and '**persis**' may also appear inside a conditional statement (**belial**/**belsa**/**elsa**) body, provided that conditional is enclosed by a loop, as in Python's conditional break/continue.
    a. When executed inside a conditional, the statement still targets the nearest enclosing loop. The conditional block itself is never a target. Its only difference from direct use in the loop body is that it runs conditionally.
    b. A '**paus**' or '**persis**' inside a conditional that is not enclosed by any loop will result in an error.

Valid Examples	Invalid Examples

cycla (x < 10) {
x += 1;
belial (x == 5) { paus; }
prin(x);
}
	cycla (x < 10) { x += 1; }
	paus; (loop control outside a loop)

lood (m insid nums) {
belial (m == 0) { persis; }
prin(m);
}
	paus (missing semicolon)

x = 0;
cycla (x < 10) {
  x += 1;
	x = 0;
	belial (x == 5) {
	  paus;



belial (x > 4) {
belial (x == 7) {
paus;
}
}
}
prin(x);
	} (not inside a loop)

M. Functions

A function is a reusable, named block of code that is declared once and invoked by name:

M.1 Function Declaration

1. A function declaration is initiated with an explicit return type (a data type keyword), immediately followed by the function name.
    a. A missing or non-type return keyword is an error.
    b. List of allowed data type keywords:
        i. **wharfa**
        ii. **flo**
        iii. **strina**
        iv. **trua**
        v. **lis**
        vi. **glossare**
2. The function name must follow identifier rules, with a minimum of three characters.
    a. A shorter name is a syntax error at the declaration.
3. The parameter list is enclosed in parentheses; a function with no parameters uses empty parentheses.
4. The parameter list contains only comma-separated <type> <identifier> pairs.
    a. A '=' inside the parameter list is a syntax error; default parameter values are not supported. Because they are not supported, neither an implicit-typed default (x=5) nor an explicit-typed (**wharfa** x=5) default is valid; both are syntax errors.
    b. Because no defaults exist, every call must supply exactly as many arguments as the function has parameters.
5. The function body must be enclosed in curly braces{}, and the declaration ends with the closing brace.
6. Each parameter is local to the function body. It is visible only inside that function.



7. A function must be declared before it is called; the point of enforcement depends on where the call appears:
    a. Calling an undeclared function, a name never declared as a function anywhere in the file is a compile-time error, whether the call is top-level or nested.
    b. A top-level call that was written outside every function body and whose callee's declaration appears later in the source file is a forward call and is a compile-time error: the compiler rejects the program before it runs.
    c. A nested call that is written inside a function body and whose callee is declared later in the file is accepted at compile time but checked at run time, when the call executes: the callee's declaration must already have been executed in top-to-bottom order and its name registered at that point. A declaration inside a branch that did not run does not count.
    d. Recursion remains valid because a function registers its own name when its declaration executes, before any call to it, including one from its own body, executes.
8. A function identifier may be declared only once.
9. Functions are declared only at the top level, not inside another function, **belial** or loop. Other top-level statements may appear between function declarations.
    a. A function declaration inside a function body, a **belial**/**belsa**/**elsa** block, or a **cycla**/**lood** block is a syntax error.

Syntax

<return_type><identifier>(<type><parameter>,<type><parameter>,...){
<statement/s>
}

Valid Examples	**Invalid Examples**

wharfa square(wharfa n){
  reor n * n;
}
	hello(){ reor "Hi"; } (no return type)

flo half(flo n){
  reor n / 2.0;
}
	flo half(flo n){ reor 2; } (wharfa return in flo function)

strina greet(){
  reor "Hi";
}
	wharfa ab(wharfa a){ reor a;} (function name under 3 characters)



wharfa myFunc(wharfa x){
reor x;
}
	wharfa myFunc(wharfa x= 5){ reor x; } (explicit-typed default)

wharfa myFunc(wharfa x){
reor x;
}
prin(myFunc(5));
	wharfa myFunc(x= 5){ reor x; } (implicit-typed default)

wharfa show(wharfa n){
prin(n); reor 0;
}
	wharfa show(wharfa n){
	prin(n); reor;
	}
	(bare reor)

	wharfa abc(a){ reor a; } (parameter missing data type)

M.2 Parameters

1. Each parameter consists of a data type and an identifier.
2. A function may have zero or more parameters.
3. Multiple parameters must be separated by commas.
4. Parameters follow identifier rules and serve as the names through which the body receives argument values.
5. At call time, arguments must match parameters in count, order, and type.
    a. A mismatch in argument count is an error.
    b. Promotion does not apply at the call boundary: a **wharfa** argument for a **flo** parameter, or a **flo** argument for a **wharfa** parameter, is an error.
    c. An argument whose type differs from its parameter's type is an error.
6. Each argument must be a literal, variable, or expression that guarantees a value.
7. A parameter declaration is exactly one data-type keyword followed by one identifier; nothing else may appear in the pair.
8. In a call, argument slots accept expressions only. Assignment and augmented assignment are statements and are never valid arguments.
    a. Assignment appears only in the init/step clauses of a counted **lood** header, never in a function header or argument list.
9. A lis or glossare argument links the parameter to the same value object as the argument. Element or entry assignment inside the body is visible to the caller after the call. Whole-value reassignment of the parameter rebinds only the parameter.

Syntax



(<type><parameter>,<type><parameter>,...)
()

Valid Examples	Invalid Examples
wharfa add(wharfa a, wharfa b){ reor a+ b; }	wharfa add(wharfa a wharfa b){ reor a+ b;} (missing comma between parameters)
strina greet(){ reor "Hi"; }	wharfa add(wharfa a,){ reor a; } (trailing comma in parameter list)
wharfa add(wharfa a, wharfa b){ reor a+ b; } prin(add(1, 2));	wharfa add(wharfa a, wharfa a){ reor a; } (duplicate parameter name)
flo half(flo n){ reor n/ 2.0; } half(2.5);	flo half(flo n){ reor n/ 2.0; } half(2); (wharfa argument to flo parameter; no promotion at call boundary)
x= 1; wharfa add(wharfa a, wharfa b){ reor a+ b; } add(x+ 1, 2);	wharfa add(wharfa a, wharfa b){ reor a+ b; } add(1); (argument count mismatch)
wharfa pair(wharfa a, strina b){ reor a; } pair(1, "x");	wharfa pair(wharfa a, strina b){ reor a; } pair("x", 1); (arguments out of order for parameter types)
wharfa bump(lis v){ v[0]+=1; reor v[0];} nums=[1,2]; bump(nums); prin(nums[0]);	x= 1; wharfa add(wharfa a, wharfa b){ reor a+ b; } add(x+= 1, 2); (augmented assignment as argument)

M.3 Return Values

1. '**reor** <expression>;' returns a value and immediately exits the function.
    a. The expression is required: a bare 'reor;' with no expression is a syntax error.
2. If an explicit return type is given, the returned value must match that type.
    a. A returned value of a different type is an error.
    b. Promotion does not apply to **reor**: a **wharfa** expression in a **flo-returning** function, or vice versa, is an error.
3. If a function is declared with return type **wharfa**, **reor** must return a **wharfa** value. The same applies to **flo**, **strina**, **trua**, **lis**, and **glossare**.



4. A function that reaches the end of its body without returning a value produces an error.
    a. This includes every execution path: if a **belial**/**belsa**/**elsa** chain lacks a **reor** in some branch and execution reaches the closing brace, the result is an error.
5. '**reor**' may appear only inside a function body.
    a. Using **reor** outside a function is an error.
6. A function whose only purpose is output still ends with a reor of its declared type.

Syntax
reor <expression>;

Valid Examples	**Invalid Examples**
wharfa add(wharfa a, wharfa b){ reor a+ b; };	wharfa add(wharfa a, wharfa b){ prin(a+ b); }; (reaches end of body without reor)
flo mix(flo n){ reor n/ 2; };	wharfa choose(wharfa n){ belial(n> 0){ reor 1; } }; (no reor on the path where n <= 0)
wharfa ident(wharfa n){ reor n; };	wharfa id(wharfa n){ reor 1.5; }; (flo value returned from a wharfa function)
wharfa show(wharfa n){ prin(n); reor 0; };	reor 5; (reor outside a function body)

M.4 Function Calls

1. To invoke a function, its unique name must be stated along with its arguments, separated by commas and contained within parentheses '( )'.
2. A function call may appear anywhere a value of its return type is expected: value slots, conditions, output, and other expressions.
3. Arguments are evaluated before the function body executes.

Syntax
<identifier>(<argument/s>);

Valid Examples	**Invalid Examples**



prin(square(4));	total = square(3; (unbalanced parenthesis)
msg = greet();	mystery(); (function not declared)
total = square(2) + 1;	square("Word"); (strina argument passed to wharfa parameter)
half(1.0);	half(1); (wharfa argument to flo parameter)
x = 5; myFunc(x);	myFunc(x= 5); (assignment as argument)

M.5 Recursion

1. A function may call itself recursively.
2. The maximum recursion depth is 999.
    a. Exceeding 999 nested calls is a runtime error; the program crashes at the call that exceeds the limit.
3. Recursion is compatible with the registration rule: a function's declaration completes and registers its name before any call to it, including one from its own body, executes.

Valid Examples	Invalid Examples

wharfa factorial(wharfa n){
  belial(n <= 1){
  reor 1;
  }
  elsa{
  reor factorial(n - 1) * n;
  }
}
	wharfa endless(wharfa n){
	  reor endless(n + 1);
	} (Runtime Error)

M.6 Scope and Visibility

1. Usable variables and functions must be declared first to be used: every definition is registered in the symbol table in execution order, that is to say that, at the moment its declaration statement executes, top to bottom; inside a function body, this order also applies to local variables.
    a. At top level, execution order coincides with source order, so a use that textually precedes its declaration including a forward function call is a compile-time error.
    b. A name read inside a function body such as a global, or a call to another function will follow the run-time rule: it must be registered at the moment



the call executes. A declaration inside a branch that did not run does not count.

2. Pothos has two scopes:
    a. Global scope: the top level of the source file, outside every function.
    b. Function scope: the body of a function, including any belial/belsa/elsa, cycla, and lood blocks nested inside it. Conditional and loop blocks do not create their own scope; a name first assigned inside them belongs to the scope that encloses the block.
3. A name first declared at top level is global; a name first declared inside a function body is local to that function.
    a. A global is accessible from any point after its registration including the execution of its declaration statement until it is deleted; for a read inside a function body, accessibility is judged at call-execution time under the run-time rule.
    b. A local is accessible only from its point of declaration to the end of its function body, and ceases to exist when the call returns. Referencing a local outside its function, including at top level after the call, is an error.
    c. Two different functions may each declare a local of the same name; they are distinct objects.
4. Within a function body, a name resolves first to a parameter of that call, then to a local of that body, then to the global scope; Pothos has no shadowing.
    a. An assignment to a name that resolves to a parameter or to a global reassigns that object; only an assignment to a name that resolves to nothing declares a new local. For this test, a global counts only if declared before the function definition.
    b. A write inside a function to a global declared before the function definition therefore modifies that global; it never creates a second object of that name.
5. A function body may read any global declared by the time the function is called.
    a. Accessing a global that is declared only after the call is an error.
    b. The check occurs at run time, when the call executes; a declaration inside a branch that did not run does not count.
6. Global variable names, constant names, and function names are unique program-wide among active identifiers.
    a. Declaring a second object under an existing active global name, at top level or inside a function, is an error, and no scope shadows another.
    b. A name freed by deleo may be redeclared for a new non-constant variable.
    c. Reusing a parameter name in another function is allowed, provided no global identifier has that name.
7. Parameters remain local to the function in which they are declared.



Valid Examples	Invalid Examples
wharfa foo(wharfa n){ total = n; reor total; } wharfa bar(wharfa n){ total = n * 2; reor total; }	wharfa foo(wharfa n){ total = n; reor total; } foo(1); prin(total); (local referenced outside its function)

total = 0; wharfa foo(wharfa n){ total = n; reor total; }
foo(7);
prin(total);
	wharfa foo(wharfa n){ reor acc; acc = n; } (local used before its declaration)

wharfa foo(wharfa n){ acc = n; belial(n > 0){ acc += 1; } reor acc; }	wharfa foo(wharfa n){ reor late; } prin(foo(1)); late = 1; (global declared only after the call)
wharfa foo(wharfa n){ n += 1; reor n; }	
wharfa foo(){ reor g;} g= 1; prin(foo());	

N. Object Removal

1. '**deleo**' removes a declared identifier and its linked value from the program:
2. '**deleo**' must be followed by an identifier.
    a. Using **deleo** on a literal or non-identifier token is an error.
3. The identifier must exist at the point of deletion.
    a. Deleting an undeclared identifier is an error.
4. The statement must end with ';'.
5. Using a deleted identifier after deletion is invalid:
    a. once deleo executes, the identifier and its linked value are gone, and a deleted variable cannot be used again until a permitted redeclaration restores it. Because constants cannot be deleted and function names cannot be deleted at all, deletion and redeclaration concern variable identifiers only.
    b. Any reference to a deleted identifier between its deletion and such a redeclaration is an error.
    c. Only variable identifiers may be redeclared after deletion: constants cannot be deleted or redeclared, and function names cannot be deleted at all, so neither kind of name ever becomes available through deletion.
6. Constants cannot be deleted or redeclared.
    a. **deleo** applied to a constant is an error.



7. After deleo<identifier>;, the identifier is removed from the active symbol table. The name may be declared again unless the identifier is a constant.
8. Function names cannot be deleted.
    a. deleo applied to a function identifier is an error.

Syntax
deleo <identifier>;

Valid Examples	**Invalid Examples**
temp = 1; deleo temp;	deleo ghost; (undeclared identifier)

x= 1;
deleo x;
x= 2;
	deleo 5; (not an identifier)

	const K = 1; deleo K; (deleting a constant)
	x = 1; deleo x; prin(x); (reference after deletion)
	wharfa foo(wharfa n){ reor n;} deleo foo; (function deletion)

VII. REGULAR DEFINITIONS

whitespace	{space, newline, tab}
caplet	{ A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z }
lowlet	{ a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z }
letter	{ caplet, lowlet }
digit	{ 1, 2, 3, 4, 5, 6, 7, 8, 9 }
escape	{\", \' , \\ , \n , \t, \0 }
numbers	{ 0, digit }



alphanum	{ letter, numbers}
mathop	{ +, -, *, /, % }
relop_ch	{ <, >, =, ! }
assops	{=, +=, -=, *=, /=, %=}
ascii	{ any printable ascii character } {printable ASCII (32-126) + tab }
str_char	{ascii, special characters}
ml_char	{ascii, whitespace}
eof	{ end-of-file marker }
special characters	{unicode symbols not found on ascii table}
delim_space	space, tab
delim_quote	', "
delim_datatype	delim_space, newline
delim_io	delim_space, (
delim_bool	Delim_space, newline,), },], ;, , , :, mathop, relop_ch, eof
delim_logic	delim_space, (
delim_block	delim_space, newline, {
delim_stmt	delim_space, newline, ;
delim_struc	delim_space, delim_quote, newline, ; (, [, {, -
delim_deleo	delim_space
delim_const	delim_space
delim_insid	delim_space, newline, (, [, “ , ‘
delim_plus	delim_space, letter, numbers, delim_quote, (, -
delim_minus	delim_space, letter, numbers, (
delim_arithmetic	delim_space, letter, numbers, (, -



delim_oper	delim_space, letter, numbers, (, -
delim_equality	delim_space, delim_quote, letter, numbers, (, - , [, {
delim_equal	delim_space, delim_quote, letter, numbers, (, [, {, -
delim_augoper	delim_space, letter, numbers, delim_quote, (, -
delim_openparen	delim_space, letter, numbers, delim_quote, (,), [, -, {
delim_closeparen	delim_space, newline, relop_ch,),], {, }, ;, ,, mathop
delim_openbracket	delim_space, letter, numbers, delim_quote, (, [, {, -
delim_closebracket	delim_space, newline, relop_ch,), [,], }, ;, ,, mathop
delim_opencurly	delim_space, newline, letter, numbers, delim_quote
delim_closecurly	delim_space, newline, delim_quote, newline, relop_ch,),] , ;
delim_comma	delim_space, newline, letter, numbers, delim_quote, (, [, {, -
delim_dqopen	delim_space, ascii
delim_dqclose	delim_space, newline, , ,) ,] , } , ;, : , relop_ch, mathop
delim_sqopen	delim_space, ascii
delim_sqclose	delim_space, newline, , ,) ,] , } , ; , : , relop_ch, mathop
delim_colon	delim_space, newline, letter, numbers, delim_quote, - , [, {
delim_scolon	delim_space, newline, letter, } , -, /, eof
delim_slcomment	delim_space, (ascii - /), special characters, eof
delim_mlcomment_open	ascii, whitespace, eof



delim_mlcomment_close	ascii, whitespace, eof
delim_id	delim_space, newline, mathop, assops, relop_ch, [,], }, (,), ; , , , eof
delim_num	delim_space, newline, mathop, relop_ch, ; , , ,], },) , : , eof



VIII. REGULAR EXPRESSION

A. Reserved Words

Pothos	REGULAR EXPRESSION	TOKEN
1. Built-in Data Types		
strina	(s)(t)(r)(i)(n)(a)	strina
wharfa	(w)(h)(a)(r)(f)(a)	wharfa
flo	(f)(l)(o)	flo
trua	(t)(r)(u)(a)	trua
lis	(l)(i)(s)	lis
glossare	(g)(l)(o)(s)(s)(a)(r)(e)	glossare
2. Input / Output Functions		
prin()	(p)(r)(i)(n)	prin
inpub()	(i)(n)(p)(u)(b)	inpub
formo()	(f)(o)(r)(m)(o)	formo
innum()	(i)(n)(n)(u)(m)	innum
inflo()	(i)(n)(f)(l)(o)	inflo
3. Value & Truth Literals		
Inaccu	(I)(n)(a)(c)(c)(u)	Inaccu
Trua	(T)(r)(u)(a)	Trua
4. Logical Operators		
an	(a)(n)	an
othan	(o)(t)(h)(a)(n)	othan
neve	(n)(e)(v)(e)	neve
5. Structural Control Flow		
belial	(b)(e)(l)(i)(a)(l)	belial
belsa	(b)(e)(l)(s)(a)	belsa



elsa	(e)(l)(s)(a)	elsa
lood	(l)(o)(o)(d)	lood
cycla	(c)(y)(c)(l)(a)	cycla
paus	(p)(a)(u)(s)	paus
persis	(p)(e)(r)(s)(i)(s)	persis
pas	(p)(a)(s)	pas
6. Structural Definitions		
reor	(r)(e)(o)(r)	reor
7. Variable Operations		
deleo	(d)(e)(l)(e)(o)	deleo
const	(c)(o)(n)(s)(t)	const
8. Object & Collection Membership		
insid	(i)(n)(s)(i)(d)	insid

B. Reserved symbols

Pothos	Regular Expressions	Token
Arithmetic Operators		
+	(+)	+
-	(-)	-
*	(*)	*
/	(/)	/
%	(%)	%
Relational Operators		
<	(<)	<
>	(>)	>
<=	(<)(=)	<=



>=	(>)(=)	>=
==	(=)(=)	==
!=	(!)(=)	!=
Assignment and Augmented Operators		
=	(=)	=
+=	(+)(=)	+=
-=	(-)(=)	-=
*=	(*)(=)	*=
/=	(/)(=)	/=
%=	(%)(=)	%=
Grouping Symbols		
(	(()	(
)	())	)
[	([)	[
]	(])	]
{	({)	{
}	(})	}
Others		
,	(,)	,
"	(")	"
'	(')	'
:	(:)	:
;	(;)	;
--	(-)(-)	--
/--	(/)(-)(-)	/--



--/	(-)(-)(/)	--/

C. Reserved Literals

Literal	Regular expressions	Token
identifier	letter(alphanum|_)^{19}	id
integer	$(0)^{+}$ | (0)*(digit)(numbers)^{7}	wharfa_lit
float	($(0)^{+}$ | (0)*(digit)(numbers)^{7}) (.) (numbers)^{4}	flo_lit
string	"ascii*" | 'ascii*' "str_char*" | 'str_char*'	strina_lit
boolean	(T)(r)(u)(a) | (I)(n)(a)(c)(c)(u)	trua_lit
single-line comment	(-)(-)((ascii - {/})(ascii)*)?	comment_sl
multi-line comment	(/)(-)(-)(ml_char)*(-)(-)(/)	comment_ml



IX. DELIMITERS

A. RESERVED WORDS

RESERVED WORDS	DELIMITER	DEFINITION
Built-in Data Types		
strina	{delim_space, newline}	delim_datatype
wharfa	{delim_space, newline }	delim_datatype
flo	{delim_space, newline }	delim_datatype
trua	{delim_space, newline}	delim_datatype
lis	{delim_space, newline}	delim_datatype
glossare	{delim_space, newline}	delim_datatype
Input / Output Functions		
prin	{ delim_space, (}	delim_io
inpub	{ delim_space, (}	delim_io
formo	{ delim_space, (}	delim_io
innum	{ delim_space, (}	delim_io
inflo	{ delim_space, (}	delim_io
Value & Truth Literals		
Inaccu	{ delim_space,), },], ;, , , :, mathop, relop_ch, eof}	delim_bool
Trua	{ delim_space,), },], ;, , , :, mathop, relop_ch, eof}	delim_bool
Logical Operators		
an	{ delim_space, (}	delim_logic
othan	{ delim_space, (}	delim_logic
neve	{ delim_space, (}	delim_logic
Structural Control Flow		



belial	{ delim_space, (}	delim_logic
belsa	{ delim_space, (}	delim_logic
elsa	{ delim_space, newline, { }	delim_block
lood	{ delim_space, (}	delim_logic
cycla	{ delim_space, (}	delim_logic
paus	{ delim_space, newline, ; }	delim_stmt
persis	{ delim_space, newline, ; }	delim_stmt
pas	{ delim_space, newline, ; }	delim_stmt
Structural Definitions		
reor	{ delim_space, delim_quote, newline, ; (, [, {, - }	delim_struc
Variable Operations		
deleo	{ delim_space}	delim_deleo
const	{ delim_space }	delim_const
Object & Collection Membership		
insid	{ delim_space, newline, (, [, “ , ‘ , - }	delim_insid

B. RESERVED SYMBOLS

Reserved Symbols	Definition	Delimiter
Arithmetic Operators		
+	{ delim_space, letter, numbers, delim_quote, (, -}	delim_plus
-	{ delim_space, letter, numbers, (}	delim_minus
*	{ delim_space, letter, numbers, (}	delim_arithmetic
/	{ delim_space, letter, numbers, (}	delim_arithmetic



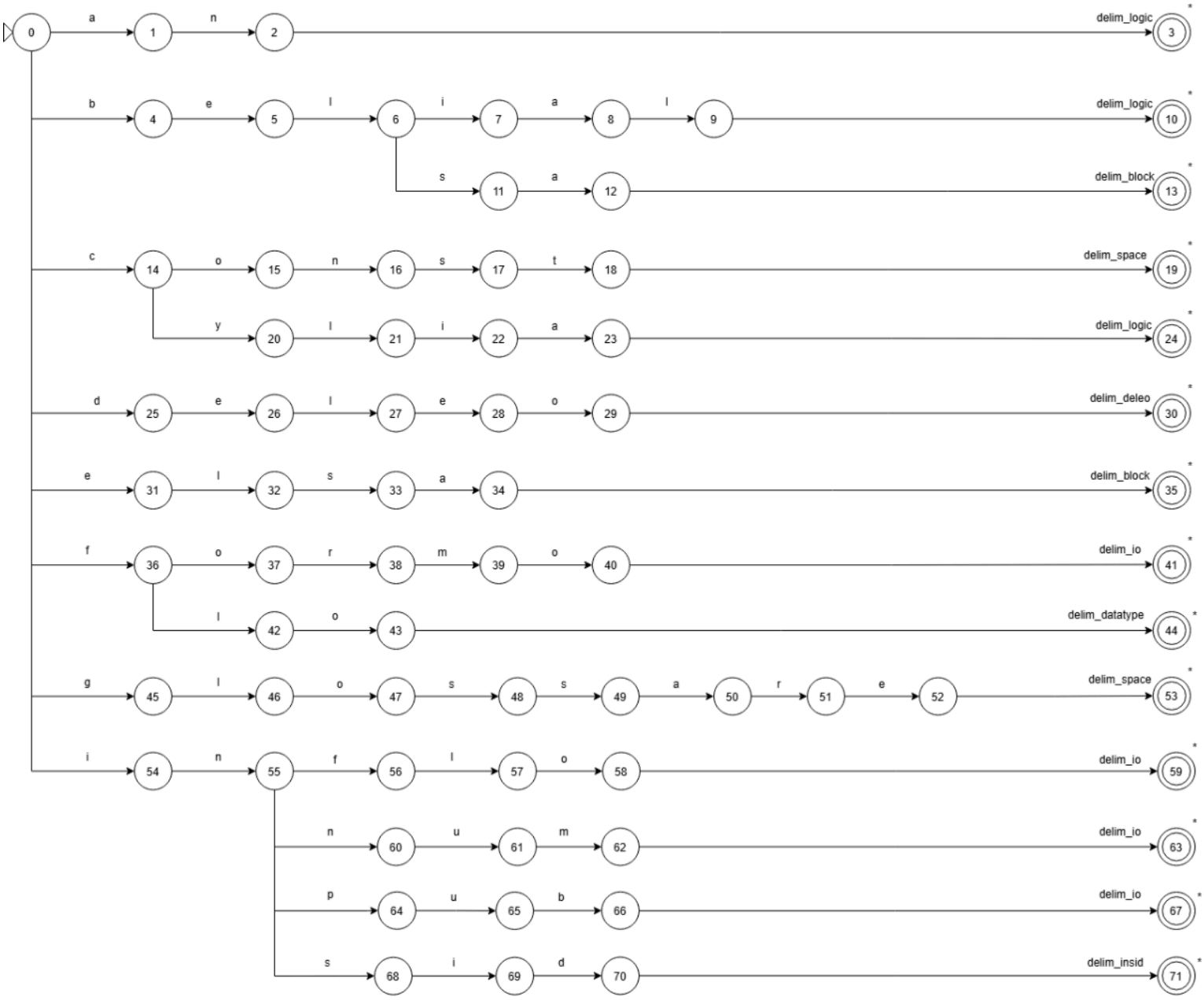
%	{ delim_space, letter, numbers, (}	delim_arithmetic
Relational Operators		
<	{ delim_space, letter, numbers, (, - }	delim_oper
>	{ delim_space, letter, numbers, (, - }	delim_oper
<=	{ delim_space, letter, numbers, (, - }	delim_oper
>=	{ delim_space, letter, numbers, (, - }	delim_oper
==	{ delim_space, delim_quote, letter, numbers, (, - , [, { }	delim_equality
!=	{ delim_space, delim_quote, letter, numbers, (, - , [, { }	delim_equality
Assignment and Augmented Operators		
=	{ delim_space, delim_quote, letter, numbers, (, [, {, - }	delim_equal
+=	{ delim_space, letter, numbers, (, - }	delim_augoper
-=	{ delim_space, letter, numbers, (, - }	delim_augoper
*=	{ delim_space, letter, numbers, (, - }	delim_augoper
/=	{ delim_space, letter, numbers, (, - }	delim_augoper
%=	{ delim_space, letter, numbers, (, - }	delim_augoper
Grouping Symbols		
(	{delim_space, letter, numbers, delim_quote, (,), [, -, {}	delim_openparen
)	{delim_space, newline, relop_ch,),], {, }, ;, ,, mathop}	delim_closeparen

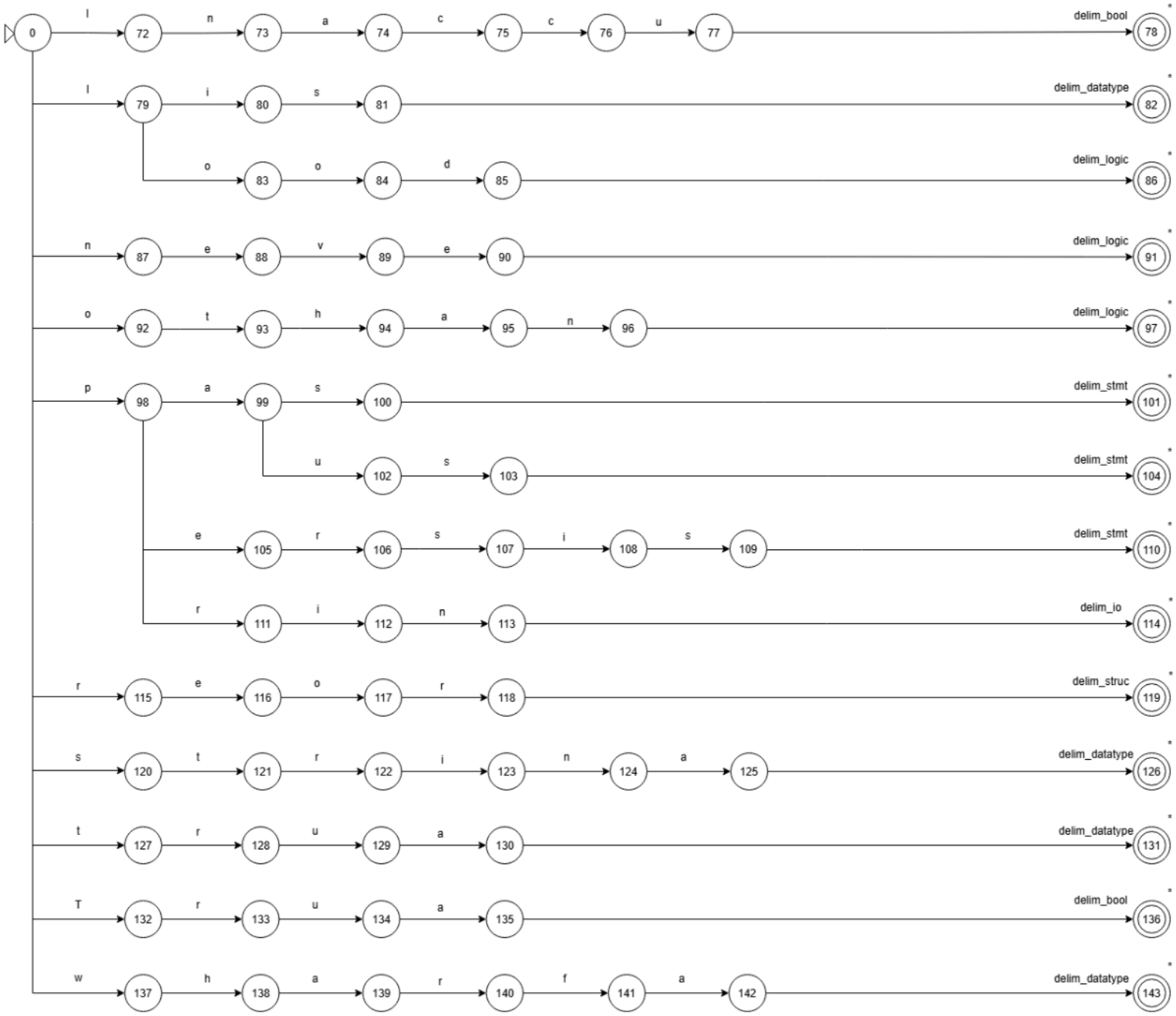


[	{delim_space, letter, numbers, delim_quote, (, [, {, - }	delim_openbracket
]	{delim_space, newline, relop_ch,), [,], }, ;, ,, mathop}	delim_closebracket
{	{delim_space, newline, letter, numbers, delim_quote }	delim_opencurly
}	{delim_space, newline, delim_quote, newline, relop_ch,),] , ;}	delim_closecurly
Others		
,	{delim_space, letter, numbers, delim_quote, (, [, {, -}	delim_comma
" (opening)	{delim_space, ascii }	delim_dqopen
" (closing)	{delim_space, newline, , ,) ,] , } , ;, : , relop_ch, mathop }	delim_dqclose
' (opening)	{delim_space, ascii }	delim_sqopen
' (closing)	{delim_space, newline, , ,) ,] , } , ; , : , relop_ch, mathop }	delim_sqclose
:	{delim_space, newline, letter, numbers, delim_quote, - , [, { }	Delim_colon
;	{delim_space, newline, letter, } , -, /, eof}	delim_scolon
--	{delim_space, ascii, special characters}	delim_slcomment
/--	{ delim_space, ascii, special characters, newline }	delim_mlcomment_open
--/	{ delim_space, letter, special characters, newline, }, eof, / }	delim_mlcomment_close
Identifiers		
{delim_space, newline, mathop, assops, relop_ch, [,], }, (,), ; , , , eof }		delim_id
{delim_space, newline, mathop, relop_ch, ; ,], },) }		delim_num

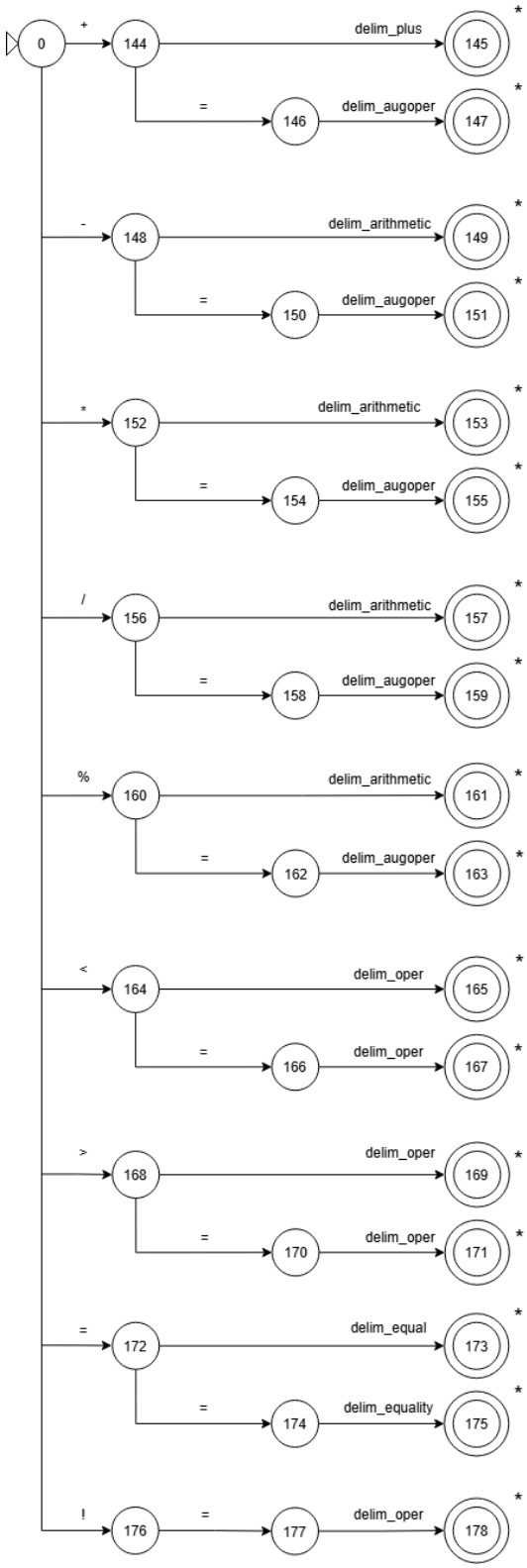
X. TRANSITION DIAGRAM

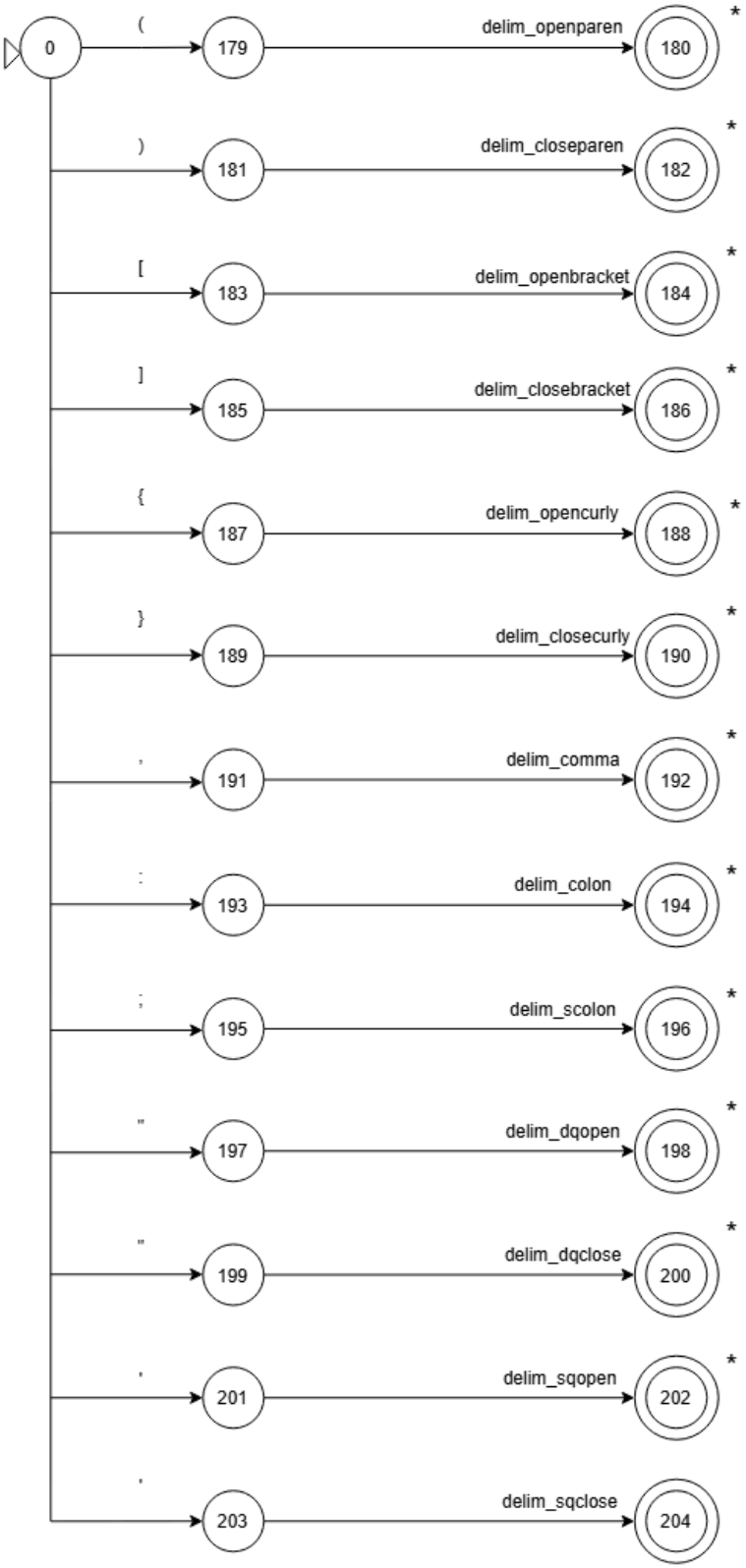
A. Reserved Words



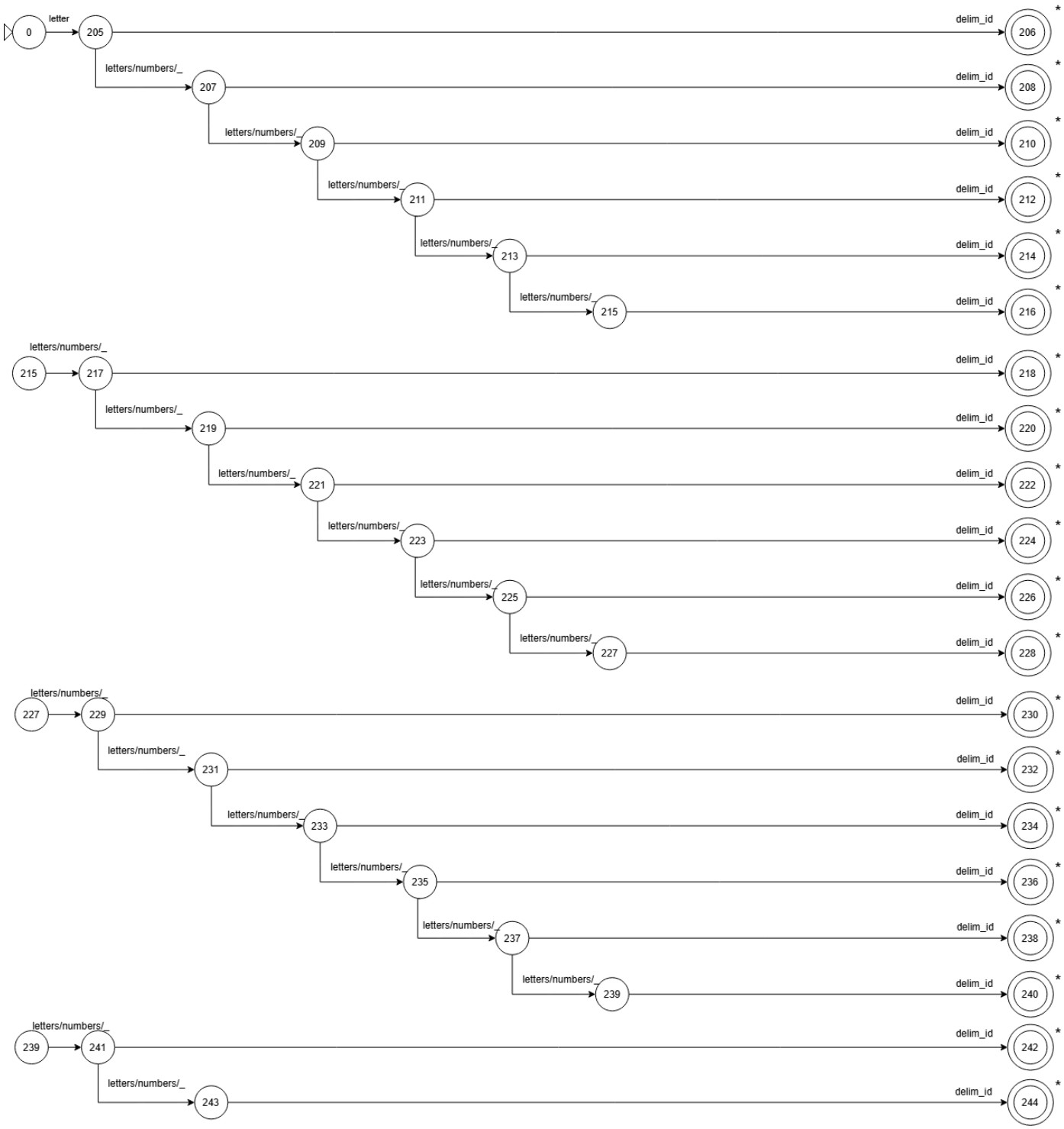

0
l
72
n
73
a
74
c
75
c
76
u
77
delim_bool
78
l
79
i
80
s
81
delim_datatype
82
o
83
o
84
d
85
delim_logic
86
n
87
e
88
v
89
e
90
delim_logic
91
o
92
t
93
h
94
a
95
n
96
delim_logic
97
p
98
a
99
s
100
delim_stmt
101
u
102
s
103
delim_stmt
104
e
105
r
106
s
107
i
108
s
109
delim_stmt
110
r
111
i
112
n
113
delim_io
114
r
115
e
116
o
117
r
118
delim_struc
119
s
120
t
121
r
122
i
123
n
124
a
125
delim_datatype
126
t
127
r
128
u
129
a
130
delim_datatype
131
T
132
r
133
u
134
a
135
delim_bool
136
w
137
h
138
a
139
r
140
f
141
a
142
delim_datatype
143

B. Reserved Symbols

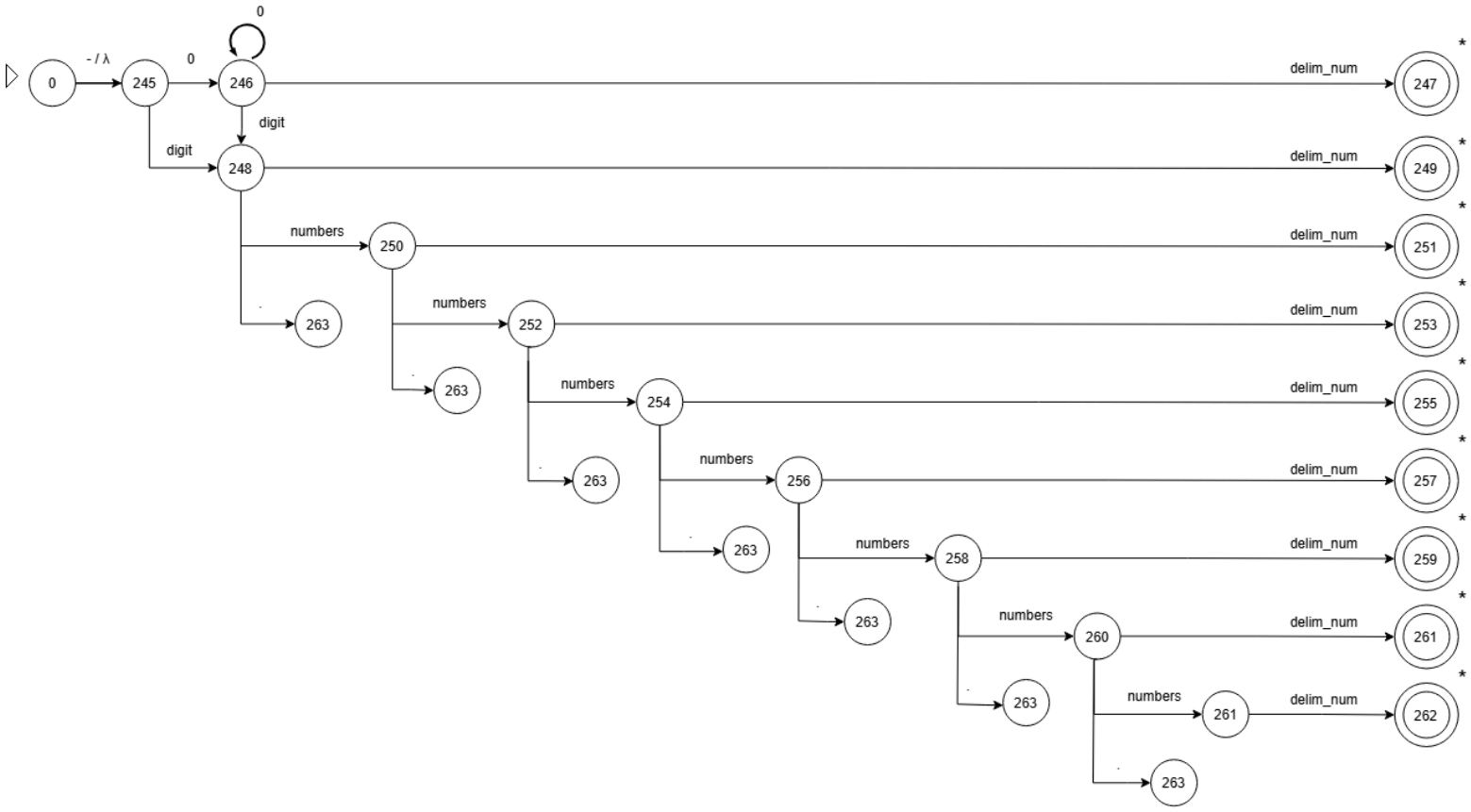


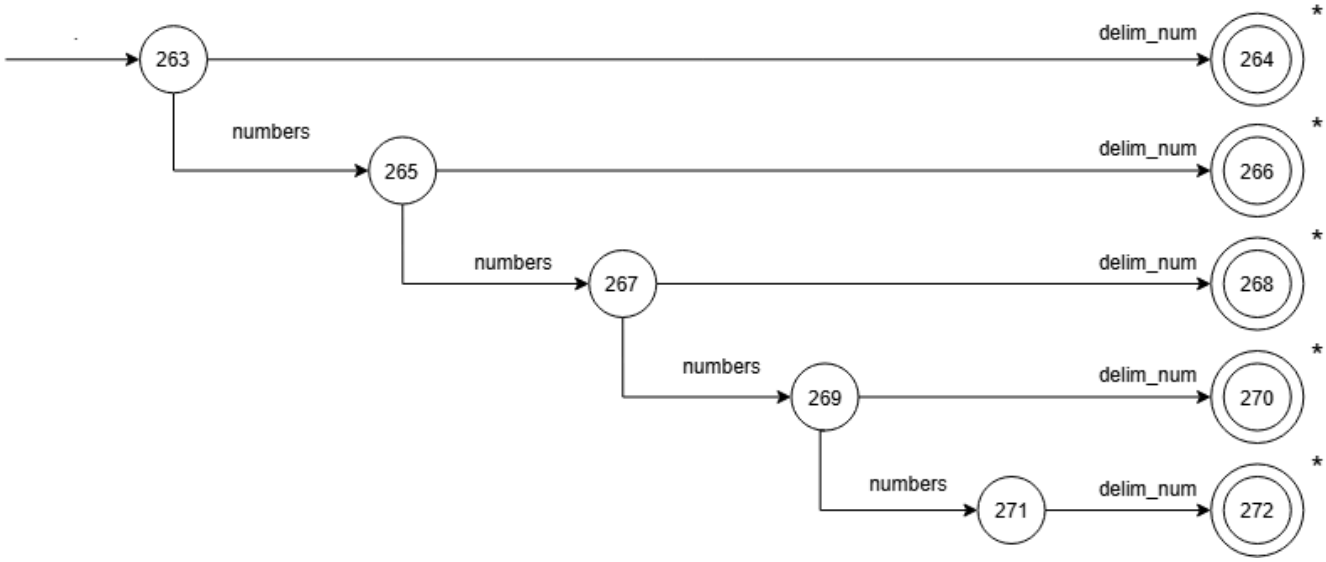

0
(
179
delim_openparen
180
*
)
181
delim_closeparen
182
*
[
183
delim_openbracket
184
*
]
185
delim_closebracket
186
*
{
187
delim_opencurly
188
*
}
189
delim_closecurly
190
*
,
191
delim_comma
192
*
:
193
delim_colon
194
*
;
195
delim_scolon
196
*
"
197
delim_dqopen
198
*
"
199
delim_dqclose
200
*
'
201
delim_sqopen
202
*
'
203
delim_sqclose
204
*

C. Identifiers

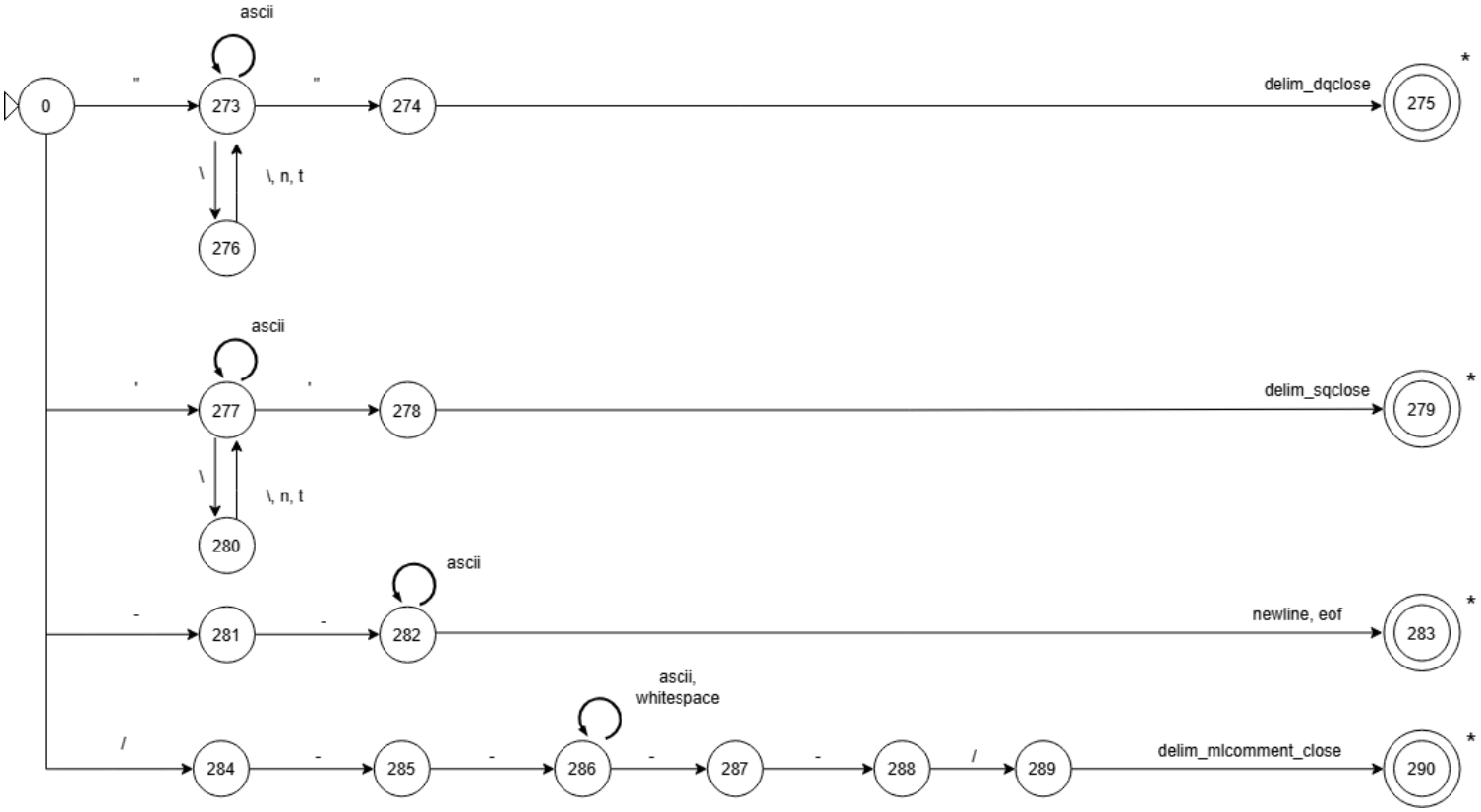


D. Integer and Float Literals





E. String Literals and Comments





CONTEXT-FREE GRAMMAR

#	Non-Terminal	→	Production Rule
1.	<program>	→	<safe_statement> <S0>
2.	<program>	→	id = <init_expression> ; <D>
3.	<program>	→	const id = <const_value> ; <D>
4.	<program>	→	trua id <trua_start_tail>
5.	<program>	→	<non_trua_function_def> <F>
6.	<S0>	→	<safe_statement> <S0>
7.	<S0>	→	id = <init_expression> ; <D>
8.	<S0>	→	const id = <const_value> ; <D>
9.	<S0>	→	trua id <trua_start_tail>
10.	<S0>	→	<non_trua_function_def> <F>
11.	<S0>	→	λ
12.	<trua_start_tail>	→	; <D>
13.	<trua_start_tail>	→	(<param_list>) <block> ; <F>
14.	<D>	→	id = <cf_expression> ; <D>
15.	<D>	→	id <aug_operator> <cf_expression> ; <D>
16.	<D>	→	const id = <const_value> ; <D>
17.	<D>	→	trua id <trua_D_tail>
18.	<D>	→	<cf_other_statement> <D>
19.	<D>	→	<non_trua_function_def> <DF>
20.	<D>	→	λ
21.	<trua_D_tail>	→	; <D>
22.	<trua_D_tail>	→	(<param_list>) <block> ; <DF>
23.	<F>	→	<safe_statement> <F>
24.	<F>	→	<non_trua_function_def> <F>



25.	<F>	→	id = <init_expression> ; <DF>
26.	<F>	→	const id = <const_value> ; <DF>
27.	<F>	→	trua id <trua_F_tail>
28.	<F>	→	λ
29.	<trua_F_tail>	→	; <DF>
30.	<trua_F_tail>	→	(<param_list>) <block> ; <F>
31.	<DF>	→	<non_trua_function_def> <DF>
32.	<DF>	→	trua id <trua_DF_tail>
33.	<DF>	→	<df_statement> <G>
34.	<DF>	→	λ
35.	<trua_DF_tail>	→	; <G>
36.	<trua_DF_tail>	→	(<param_list>) <block> ; <DF>
37.	<G>	→	<general_statement> <G>
38.	<G>	→	λ
39.	<safe_statement>	→	prin (<safe_expression>) ;
40.	<safe_statement>	→	<builtin_call> ;
41.	<safe_statement>	→	pas ;
42.	<safe_statement>	→	<safe_conditional_statement>
43.	<safe_conditional_statement>	→	belial (<safe_expression>) <safe_block> <safe_elif_tail>
44.	<safe_elif_tail>	→	<safe_belsa_clause> <safe_elif_tail>
45.	<safe_elif_tail>	→	<safe_elsa_clause>
46.	<safe_elif_tail>	→	λ
47.	<safe_belsa_clause>	→	belsa (<safe_expression>) <safe_block>
48.	<safe_elsa_clause>	→	elsa <safe_block>
49.	<safe_block>	→	{ <safe_statement_list> }
50.	<safe_statement_list>	→	<safe_statement> <safe_statement_list>



51.	<safe_statement_list>	→	λ
52.	<safe_expression>	→	<safe_othan_expression>
53.	<safe_othan_expression>	→	<safe_an_expression> <safe_othan_tail>
54.	<safe_othan_tail>	→	othan <safe_an_expression> <safe_othan_tail>
55.	<safe_othan_tail>	→	λ
56.	<safe_an_expression>	→	<safe_equality_expression> <safe_an_tail>
57.	<safe_an_tail>	→	an <safe_equality_expression> <safe_an_tail>
58.	<safe_an_tail>	→	λ
59.	<safe_equality_expression>	→	<safe_insid_expression> <safe_equality_tail>
60.	<safe_equality_tail>	→	== <safe_insid_expression> <safe_equality_tail>
61.	<safe_equality_tail>	→	!= <safe_insid_expression> <safe_equality_tail>
62.	<safe_equality_tail>	→	λ
63.	<safe_insid_expression>	→	<safe_rel_expression> <safe_insid_tail>
64.	<safe_insid_tail>	→	insid <safe_rel_expression>
65.	<safe_insid_tail>	→	λ
66.	<safe_rel_expression>	→	<safe_add_expression> <safe_rel_tail>
67.	<safe_rel_tail>	→	< <safe_add_expression> <safe_rel_tail>
68.	<safe_rel_tail>	→	> <safe_add_expression> <safe_rel_tail>
69.	<safe_rel_tail>	→	<= <safe_add_expression> <safe_rel_tail>
70.	<safe_rel_tail>	→	>= <safe_add_expression> <safe_rel_tail>
71.	<safe_rel_tail>	→	λ
72.	<safe_add_expression>	→	<safe_mul_expression> <safe_add_tail>
73.	<safe_add_tail>	→	+ <safe_mul_expression> <safe_add_tail>
74.	<safe_add_tail>	→	- <safe_mul_expression> <safe_add_tail>
75.	<safe_add_tail>	→	λ
76.	<safe_mul_expression>	→	<safe_unary_expression> <safe_mul_tail>



77.	<safe_mul_tail>	→	* <safe_unary_expression> <safe_mul_tail>
78.	<safe_mul_tail>	→	/ <safe_unary_expression> <safe_mul_tail>
79.	<safe_mul_tail>	→	% <safe_unary_expression> <safe_mul_tail>
80.	<safe_mul_tail>	→	λ
81.	<safe_unary_expression>	→	- <safe_unary_expression>
82.	<safe_unary_expression>	→	neve <safe_unary_expression>
83.	<safe_unary_expression>	→	<safe_postfix_expression>
84.	<safe_postfix_expression>	→	<builtin_call>
85.	<safe_postfix_expression>	→	(<safe_expression>)
86.	<safe_postfix_expression>	→	<literal>
87.	<safe_postfix_expression>	→	<safe_lis_literal>
88.	<safe_postfix_expression>	→	<safe_glossare_literal>
89.	<safe_lis_literal>	→	[<safe_lis_elements>]
90.	<safe_lis_elements>	→	<safe_expression> <safe_lis_tail>
91.	<safe_lis_tail>	→	, <safe_expression> <safe_lis_tail>
92.	<safe_lis_tail>	→	λ
93.	<safe_glossare_literal>	→	{ <safe_glossare_entries> }
94.	<safe_glossare_entries>	→	strina_lit : <safe_expression> <safe_glossare_tail>
95.	<safe_glossare_tail>	→	, strina_lit : <safe_expression> <safe_glossare_tail>
96.	<safe_glossare_tail>	→	λ
97.	<init_expression>	→	<literal>
98.	<init_expression>	→	<lis_init_literal>
99.	<init_expression>	→	<glossare_init_literal>
100.	<init_expression>	→	inflo ( )
101.	<init_expression>	→	innum ( )
102.	<init_expression>	→	inpub ( )



103.	<literal>	→	<numeric_literal>
104.	<literal>	→	<strina_literal>
105.	<literal>	→	<trua_literal>
106.	<numeric_literal>	→	wharfa_lit
107.	<numeric_literal>	→	flo_lit
108.	<strina_literal>	→	strina_lit
109.	<trua_literal>	→	Trua
110.	<trua_literal>	→	Inaccu
111.	<const_value>	→	<literal>
112.	<const_value>	→	- <numeric_literal>
113.	<const_value>	→	<lis_const_literal>
114.	<const_value>	→	<glossare_const_literal>
115.	<lis_const_literal>	→	[<const_elements>]
116.	<const_elements>	→	<const_value> <const_tail>
117.	<const_tail>	→	, <const_value> <const_tail>
118.	<const_tail>	→	λ
119.	<glossare_const_literal>	→	{ <const_entries> }
120.	<const_entries>	→	strina_lit : <const_value> <const_entry_tail>
121.	<const_entry_tail>	→	, strina_lit : <const_value> <const_entry_tail>
122.	<const_entry_tail>	→	λ
123.	<lis_init_literal>	→	[<init_elements>]
124.	<init_elements>	→	<init_expression> <init_tail>
125.	<init_tail>	→	, <init_expression> <init_tail>
126.	<init_tail>	→	λ
127.	<glossare_init_literal>	→	{ <init_entries> }
128.	<init_entries>	→	strina_lit : <init_expression> <init_entry_tail>



129.	<init_entry_tail>	→	, strina_lit : <init_expression> <init_entry_tail>
130.	<init_entry_tail>	→	λ
131.	<cf_other_statement>	→	prin (<cf_expression>) ;
132.	<cf_other_statement>	→	<builtin_call> ;
133.	<cf_other_statement>	→	<cf_conditional_statement>
134.	<cf_other_statement>	→	<cf_loop_statement>
135.	<cf_other_statement>	→	pas ;
136.	<cf_other_statement>	→	deleo id ;
137.	<cf_conditional_statement>	→	belial (<cf_expression>) <cf_block> <cf_elif_tail>
138.	<cf_elif_tail>	→	<cf_belsa_clause> <cf_elif_tail>
139.	<cf_elif_tail>	→	<cf_elsa_clause>
140.	<cf_elif_tail>	→	λ
141.	<cf_belsa_clause>	→	belsa (<cf_expression>) <cf_block>
142.	<cf_elsa_clause>	→	elsa <cf_block>
143.	<cf_loop_statement>	→	<cf_cycla_loop>
144.	<cf_loop_statement>	→	<cf_lood_loop>
145.	<cf_cycla_loop>	→	cycla (<cf_expression>) <cf_block>
146.	<cf_lood_loop>	→	lood (<cf_lood_header>) <cf_block>
147.	<cf_lood_header>	→	<cf_sequence_header>
148.	<cf_lood_header>	→	<cf_counted_header>
149.	<cf_sequence_header>	→	id insid <cf_expression>
150.	<cf_counted_header>	→	<cf_loop_init> ; <cf_expression> ; <cf_loop_step>
151.	<cf_loop_init>	→	<cf_simple_assignment>
152.	<cf_loop_step>	→	<cf_simple_assignment>
153.	<cf_loop_step>	→	<cf_augmented_assignment>
154.	<cf_simple_assignment>	→	id = <cf_expression>



155.	<cf_augmented_assignment>	→	id <aug_operator> <cf_expression>
156.	<cf_block>	→	{ <cf_statement_list> }
157.	<cf_statement_list>	→	id = <cf_expression> ; <cf_statement_list>
158.	<cf_statement_list>	→	id <aug_operator> <cf_expression> ; <cf_statement_list>
159.	<cf_statement_list>	→	const id = <const_value> ; <cf_statement_list>
160.	<cf_statement_list>	→	trua id ; <cf_statement_list>
161.	<cf_statement_list>	→	<cf_other_statement> <cf_statement_list>
162.	<cf_statement_list>	→	paus ; <cf_statement_list>
163.	<cf_statement_list>	→	persis ; <cf_statement_list>
164.	<cf_statement_list>	→	λ
165.	<cf_expression>	→	<cf_othan_expression>
166.	<cf_othan_expression>	→	<cf_an_expression> <cf_othan_tail>
167.	<cf_othan_tail>	→	othan <cf_an_expression> <cf_othan_tail>
168.	<cf_othan_tail>	→	λ
169.	<cf_an_expression>	→	<cf_equality_expression> <cf_an_tail>
170.	<cf_an_tail>	→	an <cf_equality_expression> <cf_an_tail>
171.	<cf_an_tail>	→	λ
172.	<cf_equality_expression>	→	<cf_insid_expression> <cf_equality_tail>
173.	<cf_equality_tail>	→	== <cf_insid_expression> <cf_equality_tail>
174.	<cf_equality_tail>	→	!= <cf_insid_expression> <cf_equality_tail>
175.	<cf_equality_tail>	→	λ
176.	<cf_insid_expression>	→	<cf_rel_expression> <cf_insid_tail>
177.	<cf_insid_tail>	→	insid <cf_rel_expression>
178.	<cf_insid_tail>	→	λ
179.	<cf_rel_expression>	→	<cf_add_expression> <cf_rel_tail>
180.	<cf_rel_tail>	→	< <cf_add_expression> <cf_rel_tail>



181.	<cf_rel_tail>	→	> <cf_add_expression> <cf_rel_tail>
182.	<cf_rel_tail>	→	<= <cf_add_expression> <cf_rel_tail>
183.	<cf_rel_tail>	→	>= <cf_add_expression> <cf_rel_tail>
184.	<cf_rel_tail>	→	λ
185.	<cf_add_expression>	→	<cf_mul_expression> <cf_add_tail>
186.	<cf_add_tail>	→	+ <cf_mul_expression> <cf_add_tail>
187.	<cf_add_tail>	→	- <cf_mul_expression> <cf_add_tail>
188.	<cf_add_tail>	→	λ
189.	<cf_mul_expression>	→	<cf_unary_expression> <cf_mul_tail>
190.	<cf_mul_tail>	→	* <cf_unary_expression> <cf_mul_tail>
191.	<cf_mul_tail>	→	/ <cf_unary_expression> <cf_mul_tail>
192.	<cf_mul_tail>	→	% <cf_unary_expression> <cf_mul_tail>
193.	<cf_mul_tail>	→	λ
194.	<cf_unary_expression>	→	- <cf_unary_expression>
195.	<cf_unary_expression>	→	neve <cf_unary_expression>
196.	<cf_unary_expression>	→	<cf_postfix_expression>
197.	<cf_postfix_expression>	→	id <cf_postfix_tail>
198.	<cf_postfix_expression>	→	<builtin_call>
199.	<cf_postfix_expression>	→	(<cf_expression>)
200.	<cf_postfix_expression>	→	<literal>
201.	<cf_postfix_expression>	→	<cf_lis_literal>
202.	<cf_postfix_expression>	→	<cf_glossare_literal>
203.	<cf_postfix_tail>	→	[<cf_expression>] <cf_postfix_tail>
204.	<cf_postfix_tail>	→	λ
205.	<cf_lis_literal>	→	[<cf_lis_elements>]
206.	<cf_lis_elements>	→	<cf_expression> <cf_lis_tail>



207.	<cf_lis_tail>	→	, <cf_expression> <cf_lis_tail>
208.	<cf_lis_tail>	→	λ
209.	<cf_glossare_literal>	→	{ <cf_glossare_entries> }
210.	<cf_glossare_entries>	→	strina_lit : <cf_expression> <cf_glossare_tail>
211.	<cf_glossare_tail>	→	, strina_lit : <cf_expression> <cf_glossare_tail>
212.	<cf_glossare_tail>	→	λ
213.	<general_statement>	→	<statements>
214.	<df_statement>	→	<variable_assignment>
215.	<df_statement>	→	<aug_assignment>
216.	<df_statement>	→	<const_declaration>
217.	<df_statement>	→	<call_statement>
218.	<df_statement>	→	<return_statement>
219.	<df_statement>	→	<break_statement>
220.	<df_statement>	→	<continue_statement>
221.	<df_statement>	→	<pass_statement>
222.	<df_statement>	→	<deleo_statement>
223.	<df_statement>	→	<conditional_statement>
224.	<df_statement>	→	<loop_statement>
225.	<statements>	→	<variable_assignment>
226.	<statements>	→	<aug_assignment>
227.	<statements>	→	<const_declaration>
228.	<statements>	→	<bool_declaration>
229.	<statements>	→	<call_statement>
230.	<statements>	→	<return_statement>
231.	<statements>	→	<break_statement>
232.	<statements>	→	<continue_statement>



233.	<statements>	→	<pass_statement>
234.	<statements>	→	<deleo_statement>
235.	<statements>	→	<conditional_statement>
236.	<statements>	→	<loop_statement>
237.	<variable_assignment>	→	id = <expression> ;
238.	<aug_assignment>	→	id <aug_operator> <expression> ;
239.	<const_declaration>	→	const id = <const_value> ;
240.	<bool_declaration>	→	trua id ;
241.	<call_statement>	→	id (<arguments_list>) ;
242.	<call_statement>	→	prin (<expression>) ;
243.	<call_statement>	→	<builtin_call> ;
244.	<arguments_list>	→	<expression> <arguments_tail>
245.	<arguments_list>	→	λ
246.	<arguments_tail>	→	, <expression> <arguments_tail>
247.	<arguments_tail>	→	λ
248.	<builtin_call>	→	formo (strina_lit)
249.	<builtin_call>	→	inflo ( )
250.	<builtin_call>	→	innum ( )
251.	<builtin_call>	→	inpub ( )
252.	<return_statement>	→	reor <expression> ;
253.	<break_statement>	→	paus ;
254.	<continue_statement>	→	persis ;
255.	<pass_statement>	→	pas ;
256.	<deleo_statement>	→	deleo id ;
257.	<conditional_statement>	→	belial (<expression>) <block> <elif_tail>
258.	<elif_tail>	→	<belsa_clause> <elif_tail>



259.	<elif_tail>	→	<elsa_clause>
260.	<elif_tail>	→	λ
261.	<belsa_clause>	→	belsa (<expression>) <block>
262.	<elsa_clause>	→	elsa <block>
263.	<block>	→	{ <statement_list> }
264.	<statement_list>	→	<statements> <statement_list>
265.	<statement_list>	→	λ
266.	<loop_statement>	→	<cycla_loop>
267.	<loop_statement>	→	<lood_loop>
268.	<cycla_loop>	→	cycla (<expression>) <block>
269.	<lood_loop>	→	lood (<lood_header>) <block>
270.	<lood_header>	→	<sequence_header>
271.	<lood_header>	→	<counted_header>
272.	<sequence_header>	→	id insid <expression>
273.	<counted_header>	→	<loop_init> ; <expression> ; <loop_step>
274.	<loop_init>	→	<simple_assignment>
275.	<loop_step>	→	<simple_assignment>
276.	<loop_step>	→	<augmented_assignment>
277.	<simple_assignment>	→	id = <expression>
278.	<augmented_assignment>	→	id <aug_operator> <expression>
279.	<aug_operator>	→	+=
280.	<aug_operator>	→	-=
281.	<aug_operator>	→	*=
282.	<aug_operator>	→	/=
283.	<aug_operator>	→	%=
284.	<expression>	→	<othan_expression>



285.	<othan_expression>	→	<an_expression> <othan_tail>
286.	<othan_tail>	→	othan <an_expression> <othan_tail>
287.	<othan_tail>	→	λ
288.	<an_expression>	→	<equality_expression> <an_tail>
289.	<an_tail>	→	an <equality_expression> <an_tail>
290.	<an_tail>	→	λ
291.	<equality_expression>	→	<insid_expression> <equality_tail>
292.	<equality_tail>	→	== <insid_expression> <equality_tail>
293.	<equality_tail>	→	!= <insid_expression> <equality_tail>
294.	<equality_tail>	→	λ
295.	<insid_expression>	→	<rel_expression> <insid_tail>
296.	<insid_tail>	→	insid <rel_expression>
297.	<insid_tail>	→	λ
298.	<rel_expression>	→	<add_expression> <rel_tail>
299.	<rel_tail>	→	< <add_expression> <rel_tail>
300.	<rel_tail>	→	> <add_expression> <rel_tail>
301.	<rel_tail>	→	<= <add_expression> <rel_tail>
302.	<rel_tail>	→	>= <add_expression> <rel_tail>
303.	<rel_tail>	→	λ
304.	<add_expression>	→	<mul_expression> <add_tail>
305.	<add_tail>	→	+ <mul_expression> <add_tail>
306.	<add_tail>	→	- <mul_expression> <add_tail>
307.	<add_tail>	→	λ
308.	<mul_expression>	→	<unary_expression> <mul_tail>
309.	<mul_tail>	→	* <unary_expression> <mul_tail>
310.	<mul_tail>	→	/ <unary_expression> <mul_tail>



311.	<mul_tail>	→	% <unary_expression> <mul_tail>
312.	<mul_tail>	→	λ
313.	<unary_expression>	→	- <unary_expression>
314.	<unary_expression>	→	neve <unary_expression>
315.	<unary_expression>	→	<postfix_expression>
316.	<postfix_expression>	→	id <postfix_tail>
317.	<postfix_expression>	→	<builtin_call>
318.	<postfix_expression>	→	(<expression>)
319.	<postfix_expression>	→	<literal>
320.	<postfix_expression>	→	<lis_literal>
321.	<postfix_expression>	→	<glossare_literal>
322.	<postfix_tail>	→	[<expression>] <postfix_tail>
323.	<postfix_tail>	→	(<arguments_list>) <postfix_tail>
324.	<postfix_tail>	→	λ
325.	<lis_literal>	→	[<lis_elements>]
326.	<lis_elements>	→	<expression> <lis_tail>
327.	<lis_tail>	→	, <expression> <lis_tail>
328.	<lis_tail>	→	λ
329.	<glossare_literal>	→	{ <glossare_entries> }
330.	<glossare_entries>	→	<glossare_entry> <glossare_tail>
331.	<glossare_entry>	→	strina_lit : <expression>
332.	<glossare_tail>	→	, <glossare_entry> <glossare_tail>
333.	<glossare_tail>	→	λ
334.	<non_trua_function_def>	→	<non_trua_datatype> id (<param_list>) <block> ;
335.	<non_trua_datatype>	→	wharfa
336.	<non_trua_datatype>	→	flo



337.	<non_trua_datatype>	→	strina
338.	<non_trua_datatype>	→	lis
339.	<non_trua_datatype>	→	glossare
340.	<datatype>	→	wharfa
341.	<datatype>	→	flo
342.	<datatype>	→	strina
343.	<datatype>	→	trua
344.	<datatype>	→	lis
345.	<datatype>	→	glossare
346.	<param_list>	→	<param> <param_tail>
347.	<param_list>	→	λ
348.	<param_tail>	→	, <param> <param_tail>
349.	<param_tail>	→	λ
350.	<param>	→	<datatype> id

FIRST SET

#	Non-Terminal	→	FIRST Set
1	First(<program>)	→	{ prin, formo(strina_lit), inflo(), innum(), inpub(), pas, belial, id, const, trua, wharfa, flo, strina, lis, glossare }
2	First(<S0>)	→	{ prin, formo(strina_lit), inflo(), innum(), inpub(), pas, belial, id, const, trua, wharfa, flo, strina, lis, glossare, λ }
3	First(<D>)	→	{ id, const, trua, prin, formo(strina_lit), inflo(), innum(), inpub(), pas, belial, cycla, lood, deleo, wharfa, flo, strina, lis, glossare, λ }



4	First(<F>)	→	{ prin, formo(strina_lit), inflo(), innum(), inpub(), pas, belial, wharfa, flo, strina, lis, glossare, id, const, trua, λ }
5	First(<DF>)	→	{ wharfa, flo, strina, lis, glossare, trua, id, const, prin, formo(strina_lit), inflo(), innum(), inpub(), reor, paus, persis, pas, deleo, belial, cycla, lood, λ }
6	First(<G>)	→	{ id, const, trua, prin, formo(strina_lit), inflo(), innum(), inpub(), reor, paus, persis, pas, deleo, belial, cycla, lood, λ }
7	First(<trua_start_tail>)	→	{ ;, (}
8	First(<trua_D_tail>)	→	{ ;, (}
9	First(<trua_F_tail>)	→	{ ;, (}
10	First(<trua_DF_tail>)	→	{ ;, (}
11	First(<safe_statement>)	→	{ prin, formo(strina_lit), inflo(), innum(), inpub(), pas, belial }
12	First(<safe_conditional_statement>)	→	{ belial }
13	First(<safe_elif_tail>)	→	{ belsa, elsa, λ }
14	First(<safe_belsa_clause>)	→	{ belsa }
15	First(<safe_elsa_clause>)	→	{ elsa }
16	First(<safe_block>)	→	{ { }
17	First(<safe_statement_list>)	→	{ prin, formo(strina_lit), inflo(), innum(), inpub(), pas, belial, λ }
18	First(<cf_other_statement>)	→	{ prin, formo(strina_lit), inflo(), innum(), inpub(), belial, cycla, lood, pas, deleo }
19	First(<cf_conditional_statement>)	→	{ belial }



20	First(<cf_elif_tail>)	→	{ belsa, elsa, λ }
21	First(<cf_belsa_clause>)	→	{ belsa }
22	First(<cf_elsa_clause>)	→	{ elsa }
23	First(<cf_loop_statement>)	→	{ cycla, lood }
24	First(<cf_cycla_loop>)	→	{ cycla }
25	First(<cf_lood_loop>)	→	{ lood }
26	First(<cf_lood_header>)	→	{ id }
27	First(<cf_sequence_header>)	→	{ id }
28	First(<cf_counted_header>)	→	{ id }
29	First(<cf_loop_init>)	→	{ id }
30	First(<cf_loop_step>)	→	{ id }
31	First(<cf_simple_assignment>)	→	{ id }
32	First(<cf_augmented_assignment>)	→	{ id }
33	First(<cf_block>)	→	{ { }
34	First(<cf_statement_list>)	→	{ id, const, trua, prin, formo(strina_lit), inflo(), innum(), inpub(), pas, belial, cycla, lood, deleo, paus, persis, λ }
35	First(<general_statement>)	→	{ id, const, trua, prin, formo(strina_lit), inflo(), innum(), inpub(), reor, paus, persis, pas, deleo, belial, cycla, lood }
36	First(<df_statement>)	→	{ id, const, trua, prin, formo(strina_lit), inflo(), innum(), inpub(), reor, paus, persis, pas, deleo, belial, cycla, lood }



37	First(<statements>)	→	{ id, const, trua, prin, formo(strina_lit), inflo(), innum(), inpub(), reor, paus, persis, pas, deleo, belial, cycla, lood }
38	First(<variable_assignment>)	→	{ id }
39	First(<aug_assignment>)	→	{ id }
40	First(<const_declaration>)	→	{ const }
41	First(<bool_declaration>)	→	{ trua }
42	First(<call_statement>)	→	{ id, prin, formo(strina_lit), inflo(), innum(), inpub() }
43	First(<arguments_list>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, {, λ }
44	First(<arguments_tail>)	→	{ ,, λ }
45	First(<return_statement>)	→	{ reor }
46	First(<break_statement>)	→	{ paus }
47	First(<continue_statement>)	→	{ persis }
48	First(<pass_statement>)	→	{ pas }
49	First(<deleo_statement>)	→	{ deleo }
50	First(<conditional_statement>)	→	{ belial }
51	First(<elif_tail>)	→	{ belsa, elsa, λ }
52	First(<belsa_clause>)	→	{ belsa }
53	First(<elsa_clause>)	→	{ elsa }
54	First(<block>)	→	{ { }



55	First(<statement_list>)	→	{ id, const, trua, prin, formo(strina_lit), inflo(), innum(), inpub(), reor, paus, persis, pas, deleo, belial, cycla, lood, λ }
56	First(<loop_statement>)	→	{ cycla, lood }
57	First(<cycla_loop>)	→	{ cycla }
58	First(<lood_loop>)	→	{ lood }
59	First(<lood_header>)	→	{ id }
60	First(<sequence_header>)	→	{ id }
61	First(<counted_header>)	→	{ id }
62	First(<loop_init>)	→	{ id }
63	First(<loop_step>)	→	{ id }
64	First(<simple_assignment>)	→	{ id }
65	First(<augmented_assignment>)	→	{ id }
66	First(<expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
67	First(<othan_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
68	First(<othan_tail>)	→	{ othan, λ }
69	First(<an_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
70	First(<an_tail>)	→	{ an, λ }
71	First(<equality_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }



72	First(<equality_tail>)	→	{ ==, !=, λ }
73	First(<insid_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
74	First(<insid_tail>)	→	{ insid, λ }
75	First(<rel_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
76	First(<rel_tail>)	→	{ <, >, <=, >=, λ }
77	First(<add_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
78	First(<add_tail>)	→	{ +, -, λ }
79	First(<mul_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
80	First(<mul_tail>)	→	{ *, /, %, λ }
81	First(<unary_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
82	First(<postfix_expression>)	→	{ id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
83	First(<postfix_tail>)	→	{ [, (, λ }
84	First(<lis_literal>)	→	{ [}
85	First(<lis_elements>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
86	First(<lis_tail>)	→	{ ,, λ }



87	First(<glossare_literal>)	→	{ { }
88	First(<glossare_entries>)	→	{ strina_lit }
89	First(<glossare_entry>)	→	{ strina_lit }
90	First(<glossare_tail>)	→	{ ,, λ }
91	First(<cf_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
92	First(<cf_othan_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
93	First(<cf_othan_tail>)	→	{ othan, λ }
94	First(<cf_an_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
95	First(<cf_an_tail>)	→	{ an, λ }
96	First(<cf_equality_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
97	First(<cf_equality_tail>)	→	{ ==, !=, λ }
98	First(<cf_insid_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
99	First(<cf_insid_tail>)	→	{ insid, λ }
100	First(<cf_rel_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
101	First(<cf_rel_tail>)	→	{ <, >, <=, >=, λ }



102	First(<cf_add_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
103	First(<cf_add_tail>)	→	{ +, -, λ }
104	First(<cf_mul_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
105	First(<cf_mul_tail>)	→	{ *, /, %, λ }
106	First(<cf_unary_expression>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
107	First(<cf_postfix_expression>)	→	{ id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
108	First(<cf_postfix_tail>)	→	{ [, λ }
109	First(<cf_lis_literal>)	→	{ [}
110	First(<cf_lis_elements>)	→	{ -, neve, id, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
111	First(<cf_lis_tail>)	→	{ ,, λ }
112	First(<cf_glossare_literal>)	→	{ { }
113	First(<cf_glossare_entries>)	→	{ strina_lit }
114	First(<cf_glossare_tail>)	→	{ ,, λ }
115	First(<safe_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
116	First(<safe_othan_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }



117	First(<safe_othan_tail>)	→	{ othan, λ }
118	First(<safe_an_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
119	First(<safe_an_tail>)	→	{ an, λ }
120	First(<safe_equality_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
121	First(<safe_equality_tail>)	→	{ ==, !=, λ }
122	First(<safe_insid_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
123	First(<safe_insid_tail>)	→	{ insid, λ }
124	First(<safe_rel_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
125	First(<safe_rel_tail>)	→	{ <, >, <=, >=, λ }
126	First(<safe_add_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
127	First(<safe_add_tail>)	→	{ +, -, λ }
128	First(<safe_mul_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
129	First(<safe_mul_tail>)	→	{ *, /, %, λ }
130	First(<safe_unary_expression>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }



131	First(<safe_postfix_expression>)	→	{ formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
132	First(<safe_lis_literal>)	→	{ [}
133	First(<safe_lis_elements>)	→	{ -, neve, formo(strina_lit), inflo(), innum(), inpub(), (, wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, { }
134	First(<safe_lis_tail>)	→	{ ,, λ }
135	First(<safe_glossare_literal>)	→	{ { }
136	First(<safe_glossare_entries>)	→	{ strina_lit }
137	First(<safe_glossare_tail>)	→	{ ,, λ }
138	First(<init_expression>)	→	{ wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, {, inflo(), innum(), inpub() }
139	First(<literal>)	→	{ wharfa_lit, flo_lit, strina_lit, Trua, Inaccu }
140	First(<numeric_literal>)	→	{ wharfa_lit, flo_lit }
141	First(<strina_literal>)	→	{ strina_lit }
142	First(<trua_literal>)	→	{ Trua, Inaccu }
143	First(<const_value>)	→	{ wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, -, [, { }
144	First(<lis_const_literal>)	→	{ [}
145	First(<const_elements>)	→	{ wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, -, [, { }
146	First(<const_tail>)	→	{ ,, λ }
147	First(<glossare_const_literal>)	→	{ { }
148	First(<const_entries>)	→	{ strina_lit }



149	First(<const_entry_tail>)	→	{ ,, λ }
150	First(<lis_init_literal>)	→	{ [}
151	First(<init_elements>)	→	{ wharfa_lit, flo_lit, strina_lit, Trua, Inaccu, [, {, inflo(), innum(), inpub() }
152	First(<init_tail>)	→	{ ,, λ }
153	First(<glossare_init_literal>)	→	{ { }
154	First(<init_entries>)	→	{ strina_lit }
155	First(<init_entry_tail>)	→	{ ,, λ }
156	First(<builtin_call>)	→	{ formo(strina_lit), inflo(), innum(), inpub() }
157	First(<aug_operator>)	→	{ +=, -=, *=, /=, %= }
158	First(<non_trua_function_def>)	→	{ wharfa, flo, strina, lis, glossare }
159	First(<non_trua_datatype>)	→	{ wharfa, flo, strina, lis, glossare }
160	First(<datatype>)	→	{ wharfa, flo, strina, trua, lis, glossare }
161	First(<param_list>)	→	{ wharfa, flo, strina, trua, lis, glossare, λ }
162	First(<param_tail>)	→	{ ,, λ }
163	First(<param>)	→	{ wharfa, flo, strina, trua, lis, glossare }

FOLLOW SET

#	Non-Terminal	→	Follow Set
1	Follow(<program>)	→	{ $ }
2	Follow(<S0>)	→	{ $ }
3	Follow(<trua_start_tail>)	→	{ $ }



4	Follow(<D>)	→	{ $ }
5	Follow(<trua_D_tail>)	→	{ $ }
6	Follow(<F>)	→	{ $ }
7	Follow(<trua_F_tail>)	→	{ $ }
8	Follow(<DF>)	→	{ $ }
9	Follow(<trua_DF_tail>)	→	{ $ }
10	Follow(<G>)	→	{ $ }
11	Follow(<safe_statement>)	→	{ belial, const, flo, formo, glossare, id, inflo, innum, inpub, lis, pas, prin, strina, trua, wharfa, }, $ }
12	Follow(<safe_conditional_statement>)	→	{ belial, const, flo, formo, glossare, id, inflo, innum, inpub, lis, pas, prin, strina, trua, wharfa, }, $ }
13	Follow(<safe_elif_tail>)	→	{ belial, const, flo, formo, glossare, id, inflo, innum, inpub, lis, pas, prin, strina, trua, wharfa, }, $ }
14	Follow(<safe_belsa_clause>)	→	{ belial, belsa, const, elsa, flo, formo, glossare, id, inflo, innum, inpub, lis, pas, prin, strina, trua, wharfa, }, $ }
15	Follow(<safe_elsa_clause>)	→	{ belial, const, flo, formo, glossare, id, inflo, innum, inpub, lis, pas, prin, strina, trua, wharfa, }, $ }
16	Follow(<safe_block>)	→	{ belial, belsa, const, elsa, flo, formo, glossare, id, inflo, innum, inpub, lis, pas, prin, strina, trua, wharfa, }, $ }
17	Follow(<safe_statement_list>)	→	{ } }
18	Follow(<safe_expression>)	→	{), ,,], } }
19	Follow(<safe_othan_expression>)	→	{), ,,], } }
20	Follow(<safe_othan_tail>)	→	{), ,,], } }
21	Follow(<safe_an_expression>)	→	{), ,,], othan, } }



22	Follow(<safe_an_tail>)	→	{), ,,], othan, } }
23	Follow(<safe_equality_expression>)	→	{), ,,], an, othan, } }
24	Follow(<safe_equality_tail>)	→	{), ,,], an, othan, } }
25	Follow(<safe_insid_expression>)	→	{ !=,), ,, ==,], an, othan, } }
26	Follow(<safe_insid_tail>)	→	{ !=,), ,, ==,], an, othan, } }
27	Follow(<safe_rel_expression>)	→	{ !=,), ,, ==,], an, insid, othan, } }
28	Follow(<safe_rel_tail>)	→	{ !=,), ,, ==,], an, insid, othan, } }
29	Follow(<safe_add_expression>)	→	{ !=,), ,, <, <=, ==, >, >=,], an, insid, othan, } }
30	Follow(<safe_add_tail>)	→	{ !=,), ,, <, <=, ==, >, >=,], an, insid, othan, } }
31	Follow(<safe_mul_expression>)	→	{ !=,), +, ,, -, <, <=, ==, >, >=,], an, insid, othan, } }
32	Follow(<safe_mul_tail>)	→	{ !=,), +, ,, -, <, <=, ==, >, >=,], an, insid, othan, } }
33	Follow(<safe_unary_expression>)	→	{ !=, %,), *, +, ,, -, /, <, <=, ==, >, >=,], an, insid, othan, } }
34	Follow(<safe_postfix_expression>)	→	{ !=, %,), *, +, ,, -, /, <, <=, ==, >, >=,], an, insid, othan, } }
35	Follow(<safe_lis_literal>)	→	{ !=, %,), *, +, ,, -, /, <, <=, ==, >, >=,], an, insid, othan, } }
36	Follow(<safe_lis_elements>)	→	{] }
37	Follow(<safe_lis_tail>)	→	{] }
38	Follow(<safe_glossare_literal>)	→	{ !=, %,), *, +, ,, -, /, <, <=, ==, >, >=,], an, insid, othan, } }



39	Follow(<safe_glossare_entries>)	→	{ } }
40	Follow(<safe_glossare_tail>)	→	{ } }
41	Follow(<init_expression>)	→	{ ,, ;,], } }
42	Follow(<literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
43	Follow(<numeric_literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
44	Follow(<strina_literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
45	Follow(<trua_literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
46	Follow(<const_value>)	→	{ ,, ;,], } }
47	Follow(<lis_const_literal>)	→	{ ,, ;,], } }
48	Follow(<const_elements>)	→	{] }
49	Follow(<const_tail>)	→	{] }
50	Follow(<glossare_const_literal>)	→	{ ,, ;,], } }
51	Follow(<const_entries>)	→	{ } }
52	Follow(<const_entry_tail>)	→	{ } }
53	Follow(<lis_init_literal>)	→	{ ,, ;,], } }
54	Follow(<init_elements>)	→	{] }
55	Follow(<init_tail>)	→	{] }
56	Follow(<glossare_init_literal>)	→	{ ,, ;,], } }
57	Follow(<init_entries>)	→	{ } }



58	Follow(<init_entry_tail>)	→	{ } }
59	Follow(<cf_other_statement>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
60	Follow(<cf_conditional_statement>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
61	Follow(<cf_elif_tail>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
62	Follow(<cf_belsa_clause>)	→	{ belial, belsa, const, cycla, deleo, elsa, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
63	Follow(<cf_elsa_clause>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
64	Follow(<cf_loop_statement>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
65	Follow(<cf_cycla_loop>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
66	Follow(<cf_lood_loop>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
67	Follow(<cf_lood_header>)	→	{) }
68	Follow(<cf_sequence_header>)	→	{) }
69	Follow(<cf_counted_header>)	→	{) }
70	Follow(<cf_loop_init>)	→	{ ; }
71	Follow(<cf_loop_step>)	→	{) }



72	Follow(<cf_simple_assignment>)	→	{), ; }
73	Follow(<cf_augmented_assignment>)	→	{) }
74	Follow(<cf_block>)	→	{ belial, belsa, const, cycla, deleo, elsa, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, strina, trua, wharfa, }, $ }
75	Follow(<cf_statement_list>)	→	{ } }
76	Follow(<cf_expression>)	→	{), ,, ;,], } }
77	Follow(<cf_othan_expression>)	→	{), ,, ;,], } }
78	Follow(<cf_othan_tail>)	→	{), ,, ;,], } }
79	Follow(<cf_an_expression>)	→	{), ,, ;,], othan, } }
80	Follow(<cf_an_tail>)	→	{), ,, ;,], othan, } }
81	Follow(<cf_equality_expression>)	→	{), ,, ;,], an, othan, } }
82	Follow(<cf_equality_tail>)	→	{), ,, ;,], an, othan, } }
83	Follow(<cf_insid_expression>)	→	{ !=,), ,, ;, ==,], an, othan, } }
84	Follow(<cf_insid_tail>)	→	{ !=,), ,, ;, ==,], an, othan, } }
85	Follow(<cf_rel_expression>)	→	{ !=,), ,, ;, ==,], an, insid, othan, } }
86	Follow(<cf_rel_tail>)	→	{ !=,), ,, ;, ==,], an, insid, othan, } }
87	Follow(<cf_add_expression>)	→	{ !=,), ,, ;, <, <=, ==, >, >=,], an, insid, othan, } }
88	Follow(<cf_add_tail>)	→	{ !=,), ,, ;, <, <=, ==, >, >=,], an, insid, othan, } }
89	Follow(<cf_mul_expression>)	→	{ !=,), +, ,, -, ;, <, <=, ==, >, >=,], an, insid, othan, } }



90	Follow(<cf_mul_tail>)	→	{ !=,), +, ,, -, ;, <, <=, ==, >, >=,], an, insid, othan, } }
91	Follow(<cf_unary_expression>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
92	Follow(<cf_postfix_expression>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
93	Follow(<cf_postfix_tail>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
94	Follow(<cf_lis_literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
95	Follow(<cf_lis_elements>)	→	{] }
96	Follow(<cf_lis_tail>)	→	{] }
97	Follow(<cf_glossare_literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
98	Follow(<cf_glossare_entries>)	→	{ } }
99	Follow(<cf_glossare_tail>)	→	{ } }
100	Follow(<general_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, $ }
101	Follow(<df_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, $ }
102	Follow(<statements>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
103	Follow(<variable_assignment>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
104	Follow(<aug_assignment>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
105	Follow(<const_declaration>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
106	Follow(<bool_declaration>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }



107	Follow(<call_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
108	Follow(<arguments_list>)	→	{) }
109	Follow(<arguments_tail>)	→	{) }
110	Follow(<builtin_call>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
111	Follow(<return_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
112	Follow(<break_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
113	Follow(<continue_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
114	Follow(<pass_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
115	Follow(<deleo_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
116	Follow(<conditional_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
117	Follow(<elif_tail>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
118	Follow(<belsa_clause>)	→	{ belial, belsa, const, cycla, deleo, elsa, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
119	Follow(<elsa_clause>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
120	Follow(<block>)	→	{ ;, belial, belsa, const, cycla, deleo, elsa, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
121	Follow(<statement_list>)	→	{ } }
122	Follow(<loop_statement>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }



123	Follow(<cycla_loop>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
124	Follow(<lood_loop>)	→	{ belial, const, cycla, deleo, formo, id, inflo, innum, inpub, lood, pas, paus, persis, prin, reor, trua, }, $ }
125	Follow(<lood_header>)	→	{) }
126	Follow(<sequence_header>)	→	{) }
127	Follow(<counted_header>)	→	{) }
128	Follow(<loop_init>)	→	{ ; }
129	Follow(<loop_step>)	→	{) }
130	Follow(<simple_assignment>)	→	{), ; }
131	Follow(<augmented_assignment>)	→	{) }
132	Follow(<aug_operator>)	→	{ (, -, Inaccu, Trua, [, flo_lit, formo, id, inflo, innum, inpub, neve, strina_lit, wharfa_lit, { }
133	Follow(<expression>)	→	{), ,, ;,], } }
134	Follow(<othan_expression>)	→	{), ,, ;,], } }
135	Follow(<othan_tail>)	→	{), ,, ;,], } }
136	Follow(<an_expression>)	→	{), ,, ;,], othan, } }
137	Follow(<an_tail>)	→	{), ,, ;,], othan, } }
138	Follow(<equality_expression>)	→	{), ,, ;,], an, othan, } }
139	Follow(<equality_tail>)	→	{), ,, ;,], an, othan, } }
140	Follow(<insid_expression>)	→	{ !=,), ,, ;, ==,], an, othan, } }
141	Follow(<insid_tail>)	→	{ !=,), ,, ;, ==,], an, othan, } }
142	Follow(<rel_expression>)	→	{ !=,), ,, ;, ==,], an, insid, othan, } }



143	Follow(<rel_tail>)	→	{ !=,), ,, ;, ==,], an, insid, othan, } }
144	Follow(<add_expression>)	→	{ !=,), ,, ;, <, <=, ==, >, >=,], an, insid, othan, } }
145	Follow(<add_tail>)	→	{ !=,), ,, ;, <, <=, ==, >, >=,], an, insid, othan, } }
146	Follow(<mul_expression>)	→	{ !=,), +, ,, -, ;, <, <=, ==, >, >=,], an, insid, othan, } }
147	Follow(<mul_tail>)	→	{ !=,), +, ,, -, ;, <, <=, ==, >, >=,], an, insid, othan, } }
148	Follow(<unary_expression>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
149	Follow(<postfix_expression >)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
150	Follow(<postfix_tail>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
151	Follow(<lis_literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
152	Follow(<lis_elements>)	→	{] }
153	Follow(<lis_tail>)	→	{] }
154	Follow(<glossare_literal>)	→	{ !=, %,), *, +, ,, -, /, ;, <, <=, ==, >, >=,], an, insid, othan, } }
155	Follow(<glossare_entries>)	→	{ } }
156	Follow(<glossare_entry>)	→	{ ,, } }
157	Follow(<glossare_tail>)	→	{ } }
158	Follow(<non_trua_function_ def>)	→	{ belial, const, cycla, deleo, flo, formo, glossare, id, inflo, innum, inpub, lis, lood, pas, paus, persis, prin, reor, strina, trua, wharfa, $ }
159	Follow(<non_trua_datatype>)	→	{ id }
160	Follow(<datatype>)	→	{ id }
161	Follow(<param_list>)	→	{) }



162	Follow(<param_tail>)	→	{) }
163	Follow(<param>)	→	{), , }